

Macacário

Dijkstravou

February 26, 2024

Contents

1	Grafos	3
1.1	Fluxo	3
1.2	Matching	6
1.3	Outros	6
1.3.1	Caminho euleriano	6
2	DP	6
2.1	Principais problemas	6
2.1.1	Knapsack 0-1	6
2.1.2	LIS	6
2.1.3	Edit Distance	6
2.2	Otimização	7
2.3	Outras técnicas	7
2.3.1	Convex Hull Trick	7
3	Geometria	7
3.1	Struct	7
3.2	Coisas gerais	8
3.2.1	Intersecção reta com círculo	8
3.3	Intersecção de círculo com círculo	8
3.4	Convex Hull	8
3.4.1	Graham Scan	8
3.4.2	Monotone Chain	9
4	Strings	9
4.1	Hashing	9
4.1.1	Rabin-Karp para matching	10
4.2	Função prefixo e KMP	10
4.3	Função Z	11
4.4	Suffix Array	12
4.5	Algoritmo de Aho-Corasick e Tries	13
4.6	Algoritmo de Ukkonen	15
4.7	Autômato de sufixos	17
4.8	Fatoração de Lyndon	17
4.9	Extras	17
4.9.1	Parsing	17
4.9.2	Algoritmo de Manacher	19
4.9.3	Encontrando repetições	20
5	Álgebra e Teoria dos Números	22
5.1	Exponenciação binária	22
5.2	MDC e MMC	22
5.3	Algoritmo de Euclides Estendido	22
5.4	Equações Diofantinas Lineares	22
5.5	Números de fibonacci	23
5.5.1	Propriedades interessantes	23
5.5.2	Fibonacci Coding	24
5.5.3	Periodicidade	24

5.5.4	Problemas	24
5.6	Primos	24
5.6.1	Crivo de Eratóstenes	24
5.6.2	Testes de primalidade	26
5.6.3	Fatoração	27
5.7	Funções comuns de teoria dos números	28
5.7.1	Função totiente de Euler	28
5.7.2	Funções quantidade e soma de divisores	28
5.7.3	Função de mobius	28
5.7.4	Funções aritméticas multiplicativas	28
5.8	Inversos modulares	29
5.9	Congruências modulares	29
5.9.1	Teorema Chinês dos Restos	30
5.9.2	Algoritmo de Garner	30
5.9.3	Fatorial módulo p	31
5.9.4	Fórmula de Legendre	31
5.9.5	Logaritmo Discreto	31
5.9.6	Raízes primitivas	32
5.9.7	Raiz discreta	33
5.10	Sistemas Numéricos	34
5.10.1	Ternário balanceado	34
5.10.2	Código de Gray	34
5.11	Manipulação de bits	34
5.11.1	Iterar por todas as submáscaras de uma máscara	35
5.12	Aritmética de precisão arbitrária	35
5.13	FFT	36
5.13.1	Number theoretic transform	36
5.13.2	FFT 2D	39
6	Combinatória	41
6.1	Fatoriais e binomiais	41
6.2	Números de catalão	42
6.3	Princípio da Inclusão e Exclusão	43
6.4	Teoria dos grupos	45
6.5	K-Combinações	46
6.6	Extras	47
7	Teoria dos Jogos	50
7.1	Jogos em grafos arbitrários	50
7.2	Teorema de Sprague-Grundy e Nim	52
7.2.1	Nim	52
7.2.2	Teorema de Sprague-Grundy	52
8	Álgebra Linear	53
8.1	Equações lineares	53
8.2	Determinante e posto	53
8.2.1	Decomposição LU	54
9	Miscelânea	56
10	Apêndice	56
10.1	Matemática	56
10.1.1	Constantes	56
10.1.2	Fórmulas e Teoremas	56
10.1.3	Números primos	56

1 Grafos

1.1 Fluxo

Problema 1.1.1: São dados um grafo $G = (V, E)$ direcionado, dois vértices s (source) e t (sink) e uma taxa máxima de fluxo (capacidade) pra cada aresta. Determinar o fluxo máximo.

Soluções:

1. Ford-Fulkerson: $O(EF)$, onde F é o fluxo máximo.
2. Edmonds-Karp: $O(VE^2)$. Ou use Dinic:
3. Dinic: $O(V^2E)$

Edmonds-Karp:

(Use uma dfs para virar Ford-Fulkerson)

```
int n;
vector<vector<int>> capacity;
vector<vector<int>> adj;

int bfs(int s, int t, vector<int>&
parent) {
    fill(parent.begin(), parent.end(),
        -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();

        for (int next : adj[cur]) {
            if (parent[next] == -1 &&
                capacity[cur][next]) {
                parent[next] = cur;
                int new_flow = min(flow
                    , capacity[cur][next]
                );
                if (next == t)
                    return new_flow;
                q.push({next, new_flow
                });
            }
        }
    }

    return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
    vector<int> parent(n);
    int new_flow;

    while (new_flow = bfs(s, t, parent)
        ) {
        flow += new_flow;
    }
}
```

```
int cur = t;
while (cur != s) {
    int prev = parent[cur];
    capacity[prev][cur] -=
        new_flow;
    capacity[cur][prev] +=
        new_flow;
    cur = prev;
}

return flow;
}
```

Dinic:

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long
        cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n),
        s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long
        long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap -
                    edges[id].flow < 1)
                    continue;
                if (level[edges[id].u]
                    != -1)
                    continue;
                level[edges[id].u] =
                    level[v] + 1;
                q.push(edges[id].u);
            }
        }
    }
}
```

```

    }
    }
    return level[t] != -1;
}

long long dfs(int v, long long
pushed) {
    if (pushed == 0)
        return 0;
    if (v == t)
        return pushed;
    for (int& cid = ptr[v]; cid < (
int)adj[v].size(); cid++) {
        int id = adj[v][cid];
        int u = edges[id].u;
        if (level[v] + 1 != level[u]
|| edges[id].cap -
edges[id].flow < 1)
            continue;
        long long tr = dfs(u, min(
pushed, edges[id].cap -
edges[id].flow));
        if (tr == 0)
            continue;
        edges[id].flow += tr;
        edges[id ^ 1].flow -= tr;
        return tr;
    }
    return 0;
}

long long flow() {
    long long f = 0;
    while (true) {
        fill(level.begin(), level.
end(), -1);
        level[s] = 0;
        q.push(s);
        if (!bfs())
            break;
        fill(ptr.begin(), ptr.end()
, 0);
        while (long long pushed =
dfs(s, flow_inf)) {
            f += pushed;
        }
    }
    return f;
}
};

```

Problema 1.1.2: $s - t - \text{cut}$ é uma partição dos vértices em um conjunto que contém s e outro t . A capacidade de um cut é a soma das capacidades das arestas indo de um conjunto pra outro. É conhecido que a capacidade do max flow é igual a do min cut. Ache esse min-cut.

Solução: Faça o max-flow. Particione em: conjunto de vértices alcançados por s no grafo residual e o resto.

Problema 1.1.3: Fluxo com demandas. Para

cada $e \in E$, queremos achar uma função f tal que $d(e) \leq f(e) \leq c(e)$ em que d, c são funções dadas.

Solução: Adicione uma source s' e um sink t' , uma aresta de s' para todos os outros vértices, uma nova aresta de cada vértice para o sink t' e uma aresta de t para s . Defina uma nova função capacidade tal que

1. $c'((s', v)) = \sum_{u \in V} d((u, v))$ para cada aresta (s', v) .
2. $c'((v, t')) = \sum_{w \in V} d((v, w))$ para cada aresta (v, t')
3. $c'((u, v)) = c((u, v)) - d((u, v))$ para cada aresta (u, v) na rede antigo.
4. $c'((t, s)) = \infty$

Se essa nova rede tiver um fluxo saturante (em que cada aresta saindo de s' está completamente preenchida, que dá no mesmo que as indo pra t' estarem), então a rede com demandas tem um fluxo válido. Basta usar algum algoritmo de max-flow para achar o saturating-flow.

Problema 1.1.4: Idêntico ao 1.1.3, mas queremos o fluxo mínimo.

Solução: Ao invés de usar $c'((t, s)) = \infty$, faça uma busca binária em $c'((t, s))$.

Problema 1.1.5: (min-cost flow) Para cada aresta, são dados a capacidade e o custo por unidade de fluxo. Encontre um fluxo de K e entre todos esses fluxos encontre o com menor custo.

Solução:

```

struct Edge
{
    int from, to, capacity, cost;
};

vector<vector<int>> adj, cost, capacity
;

const int INF = 1e9;

void shortest_paths(int n, int v0,
vector<int>& d, vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(v0);
    p.assign(n, -1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (int v : adj[u]) {
            if (capacity[u][v] > 0 && d
[v] > d[u] + cost[u][v])
            {
                d[v] = d[u] + cost[u][v]
];
            }
        }
    }
}

```

```

        p[v] = u;
        if (!inq[v]) {
            inq[v] = true;
            q.push(v);
        }
    }
}

int min_cost_flow(int N, vector<Edge>
edges, int K, int s, int t) {
    adj.assign(N, vector<int>());
    cost.assign(N, vector<int>(N, 0));
    capacity.assign(N, vector<int>(N,
0));
    for (Edge e : edges) {
        adj[e.from].push_back(e.to);
        adj[e.to].push_back(e.from);
        cost[e.from][e.to] = e.cost;
        cost[e.to][e.from] = -e.cost;
        capacity[e.from][e.to] = e.
            capacity;
    }

    int flow = 0;
    int cost = 0;
    vector<int> d, p;
    while (flow < K) {
        shortest_paths(N, s, d, p);
        if (d[t] == INF)
            break;

        // find max flow on that path
        int f = K - flow;
        int cur = t;
        while (cur != s) {
            f = min(f, capacity[p[cur]
][cur]);
            cur = p[cur];
        }

        // apply flow
        flow += f;
        cost += f * d[t];
        cur = t;
        while (cur != s) {
            capacity[p[cur]][cur] -= f;
            capacity[cur][p[cur]] += f;
            cur = p[cur];
        }

        if (flow < K)
            return -1;
        else
            return cost;
    }
}

```

Problema 1.1.6: Algoritmo húngaro com fluxo. Seleccionar n elementos de uma matriz quadrada A de

ordem n , com nenhuma fileira em comum, de modo a maximizar sua soma.

Solução:

```

const int INF = 1000 * 1000 * 1000;

vector<int> assignment(vector<vector<
int>>> a) {
    int n = a.size();
    int m = n * 2 + 2;
    vector<vector<int>>> f(m, vector<int>
>(m));
    int s = m - 2, t = m - 1;
    int cost = 0;
    while (true) {
        vector<int> dist(m, INF);
        vector<int> p(m);
        vector<bool> inq(m, false);
        queue<int> q;
        dist[s] = 0;
        p[s] = -1;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            inq[v] = false;
            if (v == s) {
                for (int i = 0; i < n;
                    ++i) {
                    if (f[s][i] == 0) {
                        dist[i] = 0;
                        p[i] = s;
                        inq[i] = true;
                        q.push(i);
                    }
                }
            } else {
                if (v < n) {
                    for (int j = n; j <
                        n + n; ++j) {
                        if (f[v][j] < 1
                            && dist[j]
                                > dist[v] +
                                    a[v][j - n])
                        {
                            dist[j] =
                                dist[v]
                                    + a[v][j
                                        - n];
                            p[j] = v;
                            if (!inq[j]
                                ) {
                                q.push(
                                    j);
                                inq[j]
                                    =
                                        true
                                    ;
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

    } else {
        for (int j = 0; j <
            n; ++j) {
            if (f[v][j] < 0
                && dist[j]
                > dist[v] -
                a[j][v - n])
            {
                dist[j] =
                    dist[v]
                    - a[j][v
                        - n];
                p[j] = v;
                if (!inq[j]
                    ) {
                    q.push(
                        j);
                    inq[j]
                        =
                            true
                    ;
                }
            }
        }
    }
}

int curcost = INF;
for (int i = n; i < n + n; ++i)
{
    if (f[i][t] == 0 && dist[i]
        < curcost) {
        curcost = dist[i];
        p[t] = i;
    }
}
if (curcost == INF)
    break;
cost += curcost;
for (int cur = t; cur != -1;
    cur = p[cur]) {
    int prev = p[cur];
    if (prev != -1)
        f[cur][prev] = -(f[prev]
            ][cur] = 1);
}

vector<int> answer(n);
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        if (f[i][j + n] == 1)
            answer[i] = j;
    }
}
return answer;
}

```

1.2 Matching

1.3 Outros

1.3.1 Caminho euleriano

2 DP

2.1 Principais problemas

2.1.1 Knapsack 0-1

53d3c9

```

// li v[MAXN], p[MAXN], dp[MAXCAP];
// li n, C;
// 0 indexado

li ks() {
    for (int i = 0; i < n; i++)
        for (li c = C; c >= p[i]; c--)
            dp[c] = max(dp[c], dp[c - p[i]] + v
                [i]);
    return dp[C];
}

```

2.1.2 LIS

4b9b86

```

// li a[MAXN], dp[MAXN];
// li n;
// 1 indexado

int lis() {
    for (int i = 1; i <= n; i++) dp[i] =
        INF;
    for (int i = 1; i <= n; i++) {
        int x = lower_bound(dp + 1, dp + 1 + n, a
            [i]) - dp - 1;
        dp[x + 1] = min(dp[x + 1], a[i]);
    }

    for (int i = n; i > 0; i--)
        if (dp[i] != INF) return i;
    return 0;
}

```

2.1.3 Edit Distance

Problema 2.1.1: Dadas strings s e t de tamanho n e m (1-indexado), pode-se fazer as seguintes operações: adicionar, alterar ou remover um caractere de uma string. Calcule a **edit distance**, que é o menor número de operações para deixar ambas strings iguais.

Solução: Note que remover um char de uma string equivale a adicionar na outra, então temos duas operações. Considere a dp de $d_{i,j}$ que é a edit distance considerando só até os caracteres i e j das strings. É fácil ver que:

$$d_{i,j} = \min\{d_{i-1,j-1} + e_{i,j}, d_{i-1,j} + 1, d_{i,j-1} + 1\}$$

Onde $e_{i,j} = 1$ se $s_i = t_j$, e é 0 caso contrário. Note que só precisa de $i - 1$ e $j - 1$ pois, para posições anteriores, terá que adicionar um caractere, e na posição a frente bastaria trocar essa adição por uma edição. A resposta é $d_{n,m}$ e a complexidade é $O(nm)$.

```

2a9da6
// string s, t
// dp[2][MAXN]

int ed_dist() {
    int c = 1;
    for(int i = 1; i <= t.size(); i++) dp[i][0] = i;

    for(int i = 1; i <= s.size(); i++, c ^= 1) {
        dp[0][c] = i;
        for(int j = 1; j <= t.size(); j++) {
            dp[j][c] = dp[j-1][c^1] + (s[i-1] != t[j-1]);
            dp[j][c] = min(dp[j][c], 1+min(dp[j][c^1], dp[j-1][c]));
        }
    }

    return dp[t.size()][c^1];
}

```

2.2 Otimização

2.3 Outras técnicas

2.3.1 Convex Hull Trick

Problema 2.3.1: Suponha que você tenha um conjunto de funções lineares, isto é, $S = \{ax + b | a, b \in \mathbb{Z}\}$. Você pode fazer duas coisas: adicionar funções ao conjunto e fazer queries que pedem, dado x , a função f que minimiza $f(x)$.

Solução: Interprete as funções como pontos (a, b) no plano. Dado uma query x , você quer encontrar o ponto (a, b) que minimiza $(x, 1) \cdot (a, b)$. É possível ver que os pontos que podem ser solução formam a parte de baixo de um fecho convexo. A intuição da resposta é pegar a reta normal a $(x, 1)$ (que tem inclinação $-x$), e ver o primeiro ponto que ela toca. As arestas do fecho estão ordenadas em relação à sua inclinação, então basta encontrar o ponto cuja inclinação das arestas passa $-x$ (isto é, a aresta anterior do ponto é menor que $-x$, e a próxima é $\geq -x$). Adicionar novos pontos deve ser trivial.

```

20376d
typedef long long int li;

struct Line {
    mutable li k, c, m;
    bool operator<(Line o) const { return k < o.k; }
    bool operator<(li x) const { return m < x; }
};

struct LineCont : multiset<Line, less<>> {
    const li inf = LLONG_MAX;
}

```

```

li div(li a, li b) { // divisao
    inteira com negativo
    return a/b - ((a^b) < 0 && a%b);
}

bool ncon(iterator a, iterator b) {
    // not convex
    if(b == end()) { a->m = inf; return 0; }
    if(a->k == b->k) a->m = (a->c > b->c) ? -inf : inf;
    else a->m = div(b->c - a->c, b->k - a->k);
    return a->m >= b->m;
}

void add(li k, li c) {
    auto z = insert({k, c, 0}), y = z++, x = y;
    while(ncon(y, z)) z = erase(z);
    if(x != begin() && ncon(--x, y)) ncon(x, erase(y));
    while((y = x) != begin() && (--x)->m >= y->m) ncon(x, erase(y));
}

li query(li x) {
    assert(!empty());
    auto l = *lower_bound(-x);
    return l.k * x + l.c;
}
};

```

3 Geometria

3.1 Struct

```

b76299
typedef long long T;

struct pt {
    T x, y;
    pt(T x=0, T y=0): x(x), y(y) {}
    pt operator+(pt o) { return pt(x+o.x, y+o.y); }
    pt operator-(pt o) { return pt(x-o.x, y-o.y); }
    //pt operator*(T a) { return pt(a*x, a*y); }
    T operator*(pt o) { return x*o.x + y*o.y; }
    T operator^(pt o) { return x*o.y - y*o.x; }
    //pt operator^(pt o) { return pt(y*o.z - z*o.y, z*o.x - x*o.z, x*o.y - y*o.x); }
    T norm2() { return x*x + y*y; }
    double norm() { return sqrt(norm2()); }
    bool operator<(pt o) const {
        if(x != o.x) return x < o.x;
        return y < o.y;
    }
}

```

```
};

// Testes insuficientes
double ang(pt a, pt b) {
    return atan2(a^b, a*b);
}

// Determinante (vetores nas linhas).
// Volume paralelepípedo.
// Usar versao 3d do cross product (^)
// T triple(pt a, pt b, pt c) { return
    a * (b^c); }
```

3.2 Coisas gerais

3.2.1 Intersecção reta com círculo

O círculo está na origem, tem raio r e a reta tem equação $ax + by + c = 0$. Ideias principais: vetor (a, b) é perpendicular à reta, então encontrar o ponto da reta mais próximo da origem (tem $d_0 = \frac{|c|}{\sqrt{a^2+b^2}}$) escalonando (a, b) para essa distância. Olhando d_0 se descobre se tem 0, 1 ou 2 intersecções. Se tem 2, os pontos são equidistantes aos encontrados, com distância $d = \sqrt{r^2 - \frac{c^2}{a^2+b^2}}$ (por Pitágoras). Esse método é mais preciso do que resolver algebricamente. EPS = precisão.

```
a9085d
int circ_line_inter(double r, double a,
    double b, double c, pt& p1, pt& p2)
{
    // Retorna a quantidade de pontos.
    // Salva em p1 e p2. Se só tem 1, p2
    // não muda

    double x0 = -a*c/(a*a+b*b), y0 = -b*c
        /(a*a+b*b);
    double dif = c*c-r*r*(a*a+b*b);
    if(dif > EPS) return 0;
    if(dif > -EPS) { // |dif| ~ 0
        p1.x = x0, p1.y = y0;
        return 1;
    }
    double d = r*r - c*c/(a*a+b*b);
    double mult = sqrt(d/(a*a+b*b));
    p1.x = x0 + b*mult;
    p2.x = x0 - b*mult;
    p1.y = y0 - a*mult;
    p2.y = y0 + a*mult;
    return 2;
}
```

3.3 Intersecção de círculo com círculo

Tem 2 dois círculos, um centrado na origem com raio r_1 , e outro no ponto x_2, y_2 com raio r_2 . Subtraindo as duas equações dos círculos, você vai reduzir ao problema de achar a intersecção da reta $(2x_2)x + (2y_2)y + (r_2^2 - r_1^2 - x_2^2 - y_2^2) = 0$ com o primeiro círculo. Só tomar cuidado caso ambos círculos estejam centrados no mesmo ponto.

3.4 Convex Hull

3.4.1 Graham Scan

```
0a1ef1
#define CW -1
#define CCW 1

int orient(pt a, pt b, pt c) {
    T v = (b-a)^(c-a);
    if(v < 0) return CW;
    else if(v > 0) return CCW;
    return 0;
}

vector<pt> convex_hull(vector<pt>& a,
    bool usecol=false) {
    // Saida em sentido horario. usecol
    // true = bota colineares no
    // resultado
    // ALERTA: garanta que nao tem pontos
    // repetidos no vetor
    // ALETA: a eh modificado (apenas
    // ordenado)
    pt p0 = *min_element(a.begin(), a.end()
        , [](pt a, pt b){
            return pt(a.y, a.x) < pt(b.y, b.x);
        });

    sort(a.begin(), a.end(), [&p0](pt a,
        pt b) {
        int o = orient(p0, a, b);
        if(o == 0) return (a - p0).norm2()
            < (b - p0).norm2();
        return o == CW;
    });

    if(usecol) {
        int i = a.size() - 2;
        while(i >= 0 && orient(p0, a[i], a.
            back()) == 0) i--;
        reverse(a.begin()+i+1, a.end());
    }

    vector<pt> st;
    for(int i = 0; i < a.size(); i++) {
        if(usecol)
            while(st.size() > 1 && orient(st[
                st.size()-2], st.back(), a[i])
                == CCW)
                st.pop_back();
            else
                while(st.size() > 1 && orient(st[
                    st.size()-2], st.back(), a[i])
                    != CW)
                    st.pop_back();
                st.push_back(a[i]);
    }

    return st;
}
```


3.4.2 Monotone Chain

2f57f6

```

#define CW -1
#define CCW 1

int orient(pt a, pt b, pt c) {
    T v = (b-a)^(c-a);
    if(v < 0) return CW;
    else if(v > 0) return CCW;
    return 0;
}

bool isor(int ot, pt a, pt b, pt c,
    bool usecol) {
    int o = orient(a, b, c);
    return o == ot or (usecol && o == 0);
}

vector<pt> convex_hull(vector<pt>& a,
    bool usecol=false) {
    // Saida em sentido horario. usecol
    // true = bota colineares no
    // resultado
    // ALERTA: garanta que nao tem pontos
    // repetidos no vetor
    // ALERTA: a eh modificado (ordenado)
    if(a.size() == 1) return a;

    sort(a.begin(), a.end());
    pt p1 = a[0], p2 = a.back();
    vector<pt> up = {p1}, down = {p2};

    for(int i = 1; i < a.size(); i++) {
        int o = CW;

        vector<pt>& t = up;
        if(i == a.size() - 1 or isor(o, p1,
            a[i], p2, usecol)) {
            while(t.size() > 1 and !isor(o, t
                [t.size() - 2], t[t.size() - 1], a
                [i], usecol))
                t.pop_back();
            t.push_back(a[i]);
        }

        { // NAO TIRAR ESSA CHAVE
            o = CCW;
            vector<pt>& t = down;
            if(i == a.size() - 1 or isor(o,
                p1, a[i], p2, usecol)) {
                while(t.size() > 1 and !isor(o,
                    t[t.size() - 2], t[t.size()
                        - 1], a[i], usecol))
                    t.pop_back();
                t.push_back(a[i]);
            }
        }
    }

    if(usecol and up.size() == a.size())

```

```

{
    reverse(a.begin(), a.end());
    return a;
}

for(int i = down.size() - 2; i > 0; i
    --) up.push_back(down[i]);
return up;
}

```

4 Strings

4.1 Hashing

Permite comparar strings em $O(1)$. Nunca é 100% deterministicamente correto. Sempre há risco de colisão.

$$\text{hash}(s) = \sum_{i=0}^{n-1} s[i] \cdot p^i \mod m$$

onde p, m são escolhidos. Essa é uma polynomial rolling hashing function. Geralmente escolhe-se p como um número perto do número de letras do alfabeto por ex $p = 31$. Se também tem uppercase usa $p = 53$. Pra minimizar colisão m deve ser muito pequeno. Na prática $m = 2^{64}$ não é recomendado. Uma boa escolha é um número primo grande por ex $m = 10^9 + 9$. Use a conversão $a \rightarrow 1, b \rightarrow 2, \dots, z \rightarrow 26$.

```

long long compute_hash(string const& s)
{
    const int p = 31;
    const int m = 1e9 + 9;
    long long hash_value = 0;
    long long p_pow = 1;
    for(char c : s) {
        hash_value = (hash_value + (c -
            'a' + 1) * p_pow) % m;
        p_pow = (p_pow * p) % m;
    }
    return hash_value;
}

```

Precomputar potências de p pode ser uma boa ideia.

Problema 4.1.1: Encontre strings duplicadas em array de strings e divida-as em grupos.

Solução: Com sort normal seria $O(nm \log n)$ mas com hash fica sendo $O(nm + n \log n)$.

```

vector<vector<int>>
group_identical_strings(vector<
    string> const& s) {
    int n = s.size();
    vector<pair<long long, int>> hashes
        (n);
    for(int i = 0; i < n; i++)
        hashes[i] = {compute_hash(s[i]),
            i};

    sort(hashes.begin(), hashes.end());

    vector<vector<int>> groups;

```

```

for (int i = 0; i < n; i++) {
    if (i == 0 || hashes[i].first
        != hashes[i-1].first)
        groups.emplace_back();
    groups.back().push_back(hashes[
        i].second);
}
return groups;
}

```

Problema 4.1.2: Calcule o hash de $s[i..j]$

Solução:

$$\text{hash}(s[i..j]) = (\text{hash}(s[0..j]) - \text{hash}(s[0..i-1])) \cdot p^{-i} \bmod$$

podemos precomputar os inversos pra ficar em $O(1)$. No entanto, ter os hashes multiplicados por p^i já é suficiente pra compará-los porque é só multiplicar o de menor potência por p^{j-i} .

Problema 4.1.3: Determine o número de substrings diferentes numa string

Solução: Iteramos por todas as substrings. PS: isso demora $O(n^2)$ então use outra coisa se quiser mesmo fazer isso!

```

int count_unique_substrings(string
const& s) {
    int n = s.size();

    const int p = 31;
    const int m = 1e9 + 9;
    vector<long long> p_pow(n);
    p_pow[0] = 1;
    for (int i = 1; i < n; i++)
        p_pow[i] = (p_pow[i-1] * p) % m;

    vector<long long> h(n + 1, 0);
    for (int i = 0; i < n; i++)
        h[i+1] = (h[i] + (s[i] - 'a' +
            1) * p_pow[i]) % m;

    int cnt = 0;
    for (int l = 1; l <= n; l++) {
        unordered_set<long long> hs;
        for (int i = 0; i <= n - l; i
            ++){
            long long cur_h = (h[i + l]
                + m - h[i]) % m;
            cur_h = (cur_h * p_pow[n-i
                -1]) % m;
            hs.insert(cur_h);
        }
        cnt += hs.size();
    }
    return cnt;
}

```

Para diminuir a probabilidade de colisão podemos utilizar dois hashes com módulos diferentes (por meio de um pair). Por exemplo $m = 10^9 + 9$ e $m = 10^9 + 7$

4.1.1 Rabin-Karp para matching

Dadas duas strings: um texto t e um padrão s , determinar se o padrão aparece no texto e se aparecer, enumerar todas as ocorrências em $O(|s| + |t|)$.

```

vector<int> rabin_karp(string const& s,
    string const& t) {
    const int p = 31;
    const int m = 1e9 + 9;
    int S = s.size(), T = t.size();

    vector<long long> p_pow(max(S, T));
    p_pow[0] = 1;
    for (int i = 1; i < (int)p_pow.size()
        ; i++)
        p_pow[i] = (p_pow[i-1] * p) % m;

    vector<long long> h(T + 1, 0);
    for (int i = 0; i < T; i++)
        h[i+1] = (h[i] + (t[i] - 'a' +
            1) * p_pow[i]) % m;
    long long h_s = 0;
    for (int i = 0; i < S; i++)
        h_s = (h_s + (s[i] - 'a' + 1) *
            p_pow[i]) % m;

    vector<int> occurrences;
    for (int i = 0; i + S - 1 < T; i++)
    {
        long long cur_h = (h[i+S] + m -
            h[i]) % m;
        if (cur_h == h_s * p_pow[i] % m)
            occurrences.push_back(i);
    }
    return occurrences;
}

```

4.2 Função prefixo e KMP

Dada uma string s de tamanho n , a função prefixo π diz para cada i qual o maior prefixo próprio (diferente da própria substring) de $s[0..i]$ que também é um sufixo dessa mesma string. Mais formalmente,

$$\pi[i] = \max_{k=0..i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Utilizamos $\pi(0) = 0$ por convenção. É possível calculá-la em $O(n)$:

```

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
}

```

```

    }
    return pi;
}

```

Algoritmo de KMP: Para procurar uma substring s num texto t , considere a string $s + \# + t$ onde $\#$ não aparece nem em s nem em t . Note que s aparece a partir de $t[i]$ se, e somente se, $\pi[2n + i] = n$, onde $n = \text{len}(s)$. Mas como o valor de π é limitado, podemos ir calculando on the fly e gastar apenas $O(n)$ de memória. Ou seja, temos $O(n + m)$ tempo e $O(n)$ memória.

Problema 4.2.1: Calcule o número de aparências de cada prefixo $s[0 \dots i]$ na string s .

Solução:

```

vector<int> ans(n + 1);
for (int i = 0; i < n; i++)
    ans[pi[i]]++;
for (int i = n - 1; i > 0; i--)
    ans[pi[i - 1]] += ans[i];
for (int i = 0; i <= n; i++)
    ans[i]++;
// Nossa resposta eh ans[i+1]

```

Problema 4.2.2: Calcule o número de ocorrências de cada prefixo $s[0 \dots i]$ em t .

Solução: Faça literalmente a mesma coisa para $s + \# + t$. Note que só estaremos interessados em $\pi[i]$ para $i \geq n + 1$.

```

vector<int> pi = prefix_function(s + "#"
    + t);
int n = s.size();
int m = t.size();
vector<int> ans(n + m + 1);
for (int i = n + 1; i < n + m + 1; i++)
    ans[pi[i]]++;
for (int i = n; i > 0; i--)
    ans[pi[i - 1]] += ans[i];
// Nossa resposta eh ans[i+1]

```

Problema 4.2.3: Número de substrings diferentes numa string

Solução: Novamente, esse não é melhor jeito, levando $O(n^2)$. Seja k a quantidade atual de substrings distintas. Colocamos um caractere c no final de s . Pra atualizar k , toma $t = s + c$, reverte t , computa o valor máximo $\pi_{\max}$ de t revertida. Atualiza $k += |s| + 1 - \pi_{\max}$.

Problema 4.2.4: Dado s de tamanho n queremos a string t de menor tamanho tal que s é uma concatenação de cópias de t

Solução: Seja $k = n - \pi[n - 1]$. Se $k|n$, então k é o tamanho da resposta, finalizando o problema. Caso contrário, não há resposta.

Note que no KMP nós concatenamos duas strings e sempre dava pra saber o π em algum índice correspondente a t on the fly. Ou seja, é possível construir um automaton i.e. uma máquina de estados finitos i.e. uma tabela de transições $(\text{old}_\pi, c) \rightarrow \text{new}_\pi$. É possível fazer isso em $O(26n)$

```

void compute_automaton(string s, vector
    <vector<int>>& aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = prefix_function(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i])
                aut[i][c] = aut[pi[i] - 1][c];
            else
                aut[i][c] = i + ('a' +
                    c == s[i]);
        }
    }
}

```

Motivos para fazer isso: acelerar o tempo para calcular a prefix function da string $s + \# + t$; t é uma string gigante construída usando umas regras.

Problema 4.2.5: Dado $k \leq 10^5$ e uma string s com tamanho $\leq 10^5$, precisamos calcular o número de ocorrências de s na k -ésima string de Gray. Lembrando que $g_1 = a$, $g_2 = aba$, $g_3 = abacaba$, etc..

Solução: Construir t é impossível. Mas só precisamos saber o valor da função de prefixos no final e, pra isso, só precisamos saber o valor dela no começo (porque daí é só ir iterando). Além da automação em si, também computamos os valores $G[i][j]$ - o valor da automação depois de processar a string g_i começando com o estado j . Também computamos os valores $K[i][j]$ - o número de ocorrências de s em g_i . Na verdade $K[i][j]$ é o número de vezes que a função prefixo tomou o valor $|s|$ enquanto as operações estavam sendo feitas. Então a resposta do problema seria $K[k][0]$.

Temos $G[0][j] = j$, $K[0][j] = 0$. Lembrando que $g_i = g_{i-1} + i + g_{i-1}$, temos

$$\text{mid} = \text{aut}[G[i - 1][j]][i]$$

$$G[i][j] = G[i - 1][\text{mid}]$$

$$K[i][j] = K[i - 1][j] + (\text{mid} == |s|) + K[i - 1][\text{mid}]$$

Problema 4.2.6: É dada s e alguns padrões t_i , sendo algum especificado assim: é uma string de caracteres normais e aí pode ter umas inserções recursivas em strings anteriores da forma t_k^{cnt} o que significa que nesse lugar devemos inserir a string t_k uma quantidade cnt de vezes. Por exemplo $t_1 = \text{"abdeca"}$, $t_2 = \text{"abc"} + t_1^{30} + \text{"abd"}$, $t_3 = t_2^{50} + t_1^{100}$ e $t_4 = t_2^{10} + t_3^{100}$.

Solução: É só fazer que nem eu disse no problema anterior.

4.3 Função Z

Seja s uma string de tamanho n . $z[i]$ é o tamanho da maior string que começa em $s[i]$ e é um prefixo de s . Escolhemos $z[0] = 0$.

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

Seja t uma string de texto e p um padrão;

Problema 4.3.1: Ache todas as ocorrências de p no texto t

Solução: Criamos a nova string $s = p + \# + t$. Computamos a função Z para s . Para cada i de 0 ate $\text{length}(t) - 1$, consideramos o valor de $k = z[i + \text{length}(p) + 1]$. Se $k == \text{length}(p)$, então sabemos que tem uma ocorrência de p na posição i de t . Com isso, o tempo e o consumo de memória são ambos $O(\text{length}(t) + \text{length}(p))$.

Problema 4.3.2: Conte o número de substrings distintas de s .

Solução: Seja k o número atual. Adicionamos c a s . Tome a string $t = s + c$ e reverta. Computa a função Z de t e acha o maior valor $z_{\max}$. O prefixo de t de tamanho $z_{\max}$ ocorre em algum lugar no meio de t também. Prefixos menores também ocorrem.

Então o número de novas substrings é $\text{length}(t) - z_{\max}$, de modo que o tempo seja $O(n^2)$.

Note que também podemos recalculer o número de substrings quando colocamos um caractere no início da string em $O(n)$.

Problema 4.3.3: Dada uma string s de tamanho n , encontre a string t de menor comprimento tal que s possa ser representada de uma ou mais cópias de t .

Solução: Computa a função Z de s , loopa pelos i tais que $i|n$ e para no primeiro i tal que $i + z[i] = n$. Então, a string s pode ser comprimida para o tamanho i .

4.4 Suffix Array

Seja s uma string de tamanho n . O i -ésimo sufixo de s é a substring $s[i \dots n - 1]$. Uma array de sufixos contém inteiros que representam os índices iniciais de todos os sufixos de uma determinada string após os sufixos serem ordenados. Por exemplo, a suffix array de $s = \text{abaab}$ é $(2, 3, 0, 4, 1)$.

Vamos primeiro fazer um algoritmo para ordenar os shifts cíclicos de uma string. A partir dele, se orden-

armos os shifts cíclicos de uma string s concatenada a um caractere estranho $\$,$ teremos a nossa suffix array em $O(n \log n)$.

```
vector<int> sort_cyclic_shifts(string
    const& s) {
    int n = s.size();
    const int alphabet = 256;
    vector<int> p(n), c(n), cnt(max(
        alphabet, n), 0);
    for (int i = 0; i < n; i++)
        cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++)
        cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; i++)
        p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i - 1]])
            classes++;
        c[p[i]] = classes - 1;
    }
    vector<int> pn(n), cn(n);
    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0)
                pn[i] += n;
        }
        fill(cnt.begin(), cnt.begin() +
            classes, 0);
        for (int i = 0; i < n; i++)
            cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]],
                c[(p[i] + (1 << h))
                    % n]};
            pair<int, int> prev = {c[p[i - 1]],
                c[(p[i - 1] + (1 <<
                    h)) % n]};
            if (cur != prev)
                ++classes;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    return p;
}
```

Isso requer tempo $O(n \log n)$ e $O(n)$ de memória. A partir dele:

```
vector<int> suffix_array_construction(
    string s) {
```

```

s += "$";
vector<int> sorted_shifts =
    sort_cyclic_shifts(s);
sorted_shifts.erase(sorted_shifts.
    begin());
return sorted_shifts;
}

```

Problema 4.4.1: Encontre o menor cyclic shift.

Solução: O algoritmo colocado ordena todos os shifts cíclicos e então $p[0]$ dá a posição do menor shift cíclico.

Problema 4.4.2: Ache uma string s dentro de um texto t online. Sabemos o texto t de antemão mas não a string s .

Solução: Fazemos a suffix array para o texto t em $O(|t| \log |t|)$. Certamente s deve ser um prefixo de um sufixo de t . Então basta fazer uma busca binária por s em p . Com isso, demoramos $O(|s| \log |t|)$. Se ocorrer mais vezes, então vão tar próximas, então faz uma segunda busca binária e acha o número de ocorrências dela.

Problema 4.4.3: Queremos saber se uma substring de s é menor que outra em $O(1)$

Solução: Depois de construir em $O(|s| \log |s|)$ a suffix array, podemos fazer em $O(1)$:

```

int compare(int i, int j, int l, int k)
{
    pair<int, int> a = {c[k][i], c[k][(i+1-(1<<k))%n]};
    pair<int, int> b = {c[k][j], c[k][(j+1-(1<<k))%n]};
    return a == b ? 0 : a < b ? -1 : 1;
}

```

Problema 4.4.4: Dada uma string s , queremos o longest common prefix de dois sufixos que iniciam nas posições i e j . (aqui fazemos com $O(|s| \log |s|)$ de memória inicial, mas no problema seguinte fazemos sem).

Solução: Temos que nos lembrar das arrays $c[]$ de cada iteração e daí vem a memória adicional.

```

//log_n e o log(n) na base 2
arredondado pra baixo
int lcp(int i, int j) {
    int ans = 0;
    for (int k = log_n; k >= 0; k--) {
        if (c[k][i % n] == c[k][j % n]) {
            ans += 1 << k;
            i += 1 << k;
            j += 1 << k;
        }
    }
    return ans;
}

```

Problema 4.4.5: Mesma coisa só que sem memória adicional.

Solução: Construa a array $lcp[0 \dots n-2]$ onde $lcp[i]$ é igual ao tamanho do longest common prefix

dos sufixos que iniciam em $p[i]$ e $p[i+1]$. Essa array dirá para nós uma resposta para quaisquer dois sufixos adjacentes dessa string. Suponha que seja o pedido de computar o LCP dos sufixos $p[i]$ e $p[j]$. A resposta será $\min(lcp[i], \dots, lcp[j-1])$. Ou seja será uma range minimum query que tem uma variedade de soluções. Para construir a lcp podemos usar o algoritmo de Kasai

```

vector<int> lcp_construction(string
const& s, vector<int> const& p) {
    int n = s.size();
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++)
        rank[p[i]] = i;

    int k = 0;
    vector<int> lcp(n-1, 0);
    for (int i = 0; i < n; i++) {
        if (rank[i] == n-1) {
            k = 0;
            continue;
        }
        int j = p[rank[i] + 1];
        while (i+k < n && j+k < n
            && s[i+k] == s[j+k])
            k++;
        lcp[rank[i]] = k;
        if (k)
            k--;
    }
    return lcp;
}

```

Problema 4.4.6: Número de substrings distintas

Solução: Pensamos em quais novas substrings começam na posição $p[0]$ depois em $p[1]$ etc.. Tomamos os sufixos ordenadamente e vemos quais prefixos dão novas substrings. Para cada sufixo atual $p[i]$ teremos novas substrings para todos seus prefixos, exceto para os prefixos que coincidirem com o sufixo $p[i-1]$. Então pegamos todos os prefixos exceto os $lcp[i-1]$ primeiros. Então a resposta final é:

$$\sum_{i=0}^{n-1} (n - p[i]) - \sum_{i=0}^{n-2} lcp[i] = \frac{n^2 + n}{2} - \sum_{i=0}^{n-2} lcp[i]$$

4.5 Algoritmo de Aho-Corasick e Tries

O algoritmo de Aho-Corasick permite uma pesquisa rápida por múltiplos padrões num texto. O conjunto de strings que são padrões é chamado de dicionário. Denota-se o tamanho total das strings constituintes por m e o tamanho do alfabeto por k . O algoritmo constrói um autômato de estados finitos baseado em um trie em tempo $O(mk)$ e usa ele para processar o texto.

Um trie é uma árvore enraizada em que cada aresta é rotulada com alguma letra.

```

const int K = 26;

struct Vertex {
    int next[K];
}

```

```

    bool output = false;

    Vertex() {
        fill(begin(next), end(next),
             -1);
    }
};

vector<Vertex> trie(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}

```

É possível trocar de array para mapa para reduzir o consumo de memória para $O(m)$, mas isso aumenta a complexidade para $O(m \log k)$.

Construímos o autômato de estados finitos da seguinte forma: Se estamos no vértice correspondente a uma string t e queremos ir para um estado diferente usando o caractere c , então veja se tem uma aresta com esse caractere. Se tiver, vá para o vértice apontado. Se não tiver, achamos a maior string do trie que é um sufixo próprio da string t e a partir daí tenta fazer a transição. Com isso queremos manter a invariante de que o estado atual é o maior match parcial na string processada.

Um suffix link para um vértice p é uma aresta que aponta para o maior sufixo próprio da string correspondendo ao vértice p . O único caso especial é a raiz da árvore, cujo suffix link apontará para ela mesma. Agora reformulamos o statement sobre as transições na automação assim: enquanto não tiver transição do vértice atual da trie usando a letra atual, seguimos o suffix link.

```

const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
}

```

```

};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0 :
                go(get_link(v), ch);
    }
    return t[v].go[c];
}

```

O tempo total de achar todos os suffix links e transições será linear.

Também é possível fazer uma construção baseada em BFS (bottom-up a partir da raiz) ao invés de ficar computando transições e suffix links com chamadas recursivas. Isso tem algumas vantagens: ao invés de tamanho total m , o running time depende apenas no número de vértices n na trie. Além disso, é possível adaptar para alfabetos grandes usando uma estrutura de dados de array persistente, tornando o tempo de construção $O(n \log k)$ ao invés de $O(mk)$ (lembrando que m pode ir até n^2).

Podemos pensar indutivamente usando o fato de que a BFS a partir da raiz atravessa vértices na ordem de increasing length. Podemos assumir que quando estamos num vértice v , seu suffix link $u = \text{link}[v]$ já foi computado com sucesso e para todos os vértices com transições de length menores a partir deles também já foram computados.

Assuma que no momento que estamos em um vértice v e consideramos um caractere c , essencialmente temos dois casos

1. $go[v][c] = -1$. Nesse caso, designamos $go[v][c] = go[u][c]$, que já é conhecido pela hipótese de indução
2. $go[v][c] = w \neq -1$. Nesse caso, designamos $link[w] = go[u][c]$

Dessa maneira, gastamos $O(1)$ de tempo para cada par vértice/caractere, tornando o running time $O(mk)$. Mas nós copiamos aqui várias transições a partir de u no primeiro caso, enquanto que transições do segundo caso formam a trie e somam up to m over all vertices. Para evitar a cópia de $go[u][c]$, podemos usar uma estrutura de dados persistente, com a qual copiamos $go[u]$ para $go[v]$ e aí atualizamos os valores para caracteres em que a transição difere. Isso leva a um algoritmo $O(m \log k)$.

Problema 4.5.1: Encontre (printe) todas as strings de um conjunto em um texto em $O(len + ans)$ onde len é o tamanho do texto e ans é o tamanho das strings printadas.

Solução: Construímos um autômato para esse conjunto de strings. Agora processamos cada letra do texto usando o autômato, começando pela raiz. Se estamos num estado v e vamos processar agora um caractere c fazemos $go(v, c)$. Como encontrar para um estado v se tem algum match com strings do conjunto?

Guardamos cada output vertex no índice da string correspondente a ele (ou então listas de índices caso strings duplicatas apareçam no conjunto). Daí encontramos em $O(n)$ os índices de todas as strings que dão match no estado atual simplesmente seguindo os suffix links a partir do vértice atual na raiz. Isso resulta em $O(nlen)$, o que não é muito bom. Dá pra otimizar computando e guardando o vértice output mais próximo que é alcançável utilizando suffix links. Isso pode ser feito linearmente de forma preguiçosa. Então pra cada vértice podemos avançar em $O(1)$ tempo para o próximo vértice marcado no caminho do suffix link i.e. pro próximo match. Então pra cada match gastamos $O(1)$ de maneira que fiquemos com $O(len + ans)$.

Problema 4.5.2: Encontrando a menor string (lexicograficamente) de um dado tamanho que não dá match em nenhuma das strings dadas.

Solução: Construa um autômato para o conjunto de strings. Como os vértices de output são os estados em que temos um match com uma string do conjunto, temos que evitar esses estados já que temos que evitar matches. Então deletamos todos os vértices ruins das máquinas e no grafo remanescente do autômato encontramos o menor caminho lexicograficamente de tamanho L . Fazemos isso em $O(L)$ com uma dfs.

Problema 4.5.3: Encontrando a menor string contendo todas as strings dadas.

Solução: Usa a mesma ideia. Para cada vértice, guarda uma máscara que denota as strings que dão match nesse estado. Então o problema pode ser reformulado assim: inicialmente, estando no estado

($v = root, mask = 0$), queremos chegar no estado ($v, mask = 2^n - 1$), em que n é o número de strings do conjunto. Vamos de um estado para outro usando uma letra e damos update na máscara de acordo. Usando uma BFS podemos encontrar qualquer caminho para esse estado com o menor tamanho.

Problema 4.5.4: Encontrando a menor string lexicograficamente de tamanho L contendo k strings.

Solução: Parecido com o outro problema. Mas agora o estado atual é determinado por uma tripla (v, len, cnt) e queremos ir do estado ($root, 0, 0$) para o estado (v, L, k), onde v pode ser qualquer vértice. Então podemos encontrar esse caminho usando dfs.

4.6 Algoritmo de Ukkonen

Constrói uma árvore de sufixos para uma string s de tamanho n em $O(n \log k)$ em que k é o tamanho do alfabeto.

A função `build_tree` constrói a árvore de sufixos. Ela é guardada num array de estruturas `node` onde `node[0]` é a raiz.

```
string s;
int n;

struct node {
    int l, r, par, link;
    map<char, int> next;

    node (int l=0, int r=0, int par=-1)
        : l(l), r(r), par(par), link
          (-1) {}
    int len() { return r - l; }
    int &get (char c) {
        if (!next.count(c)) next[c] = -1;
        return next[c];
    }
};
node t[MAXN];
int sz;

struct state {
    int v, pos;
    state (int v, int pos) : v(v), pos(
        pos) {}
};
state ptr (0, 0);

state go (state st, int l, int r) {
    while (l < r)
        if (st.pos == t[st.v].len()) {
            st = state (t[st.v].get (s[
                l]), 0);
            if (st.v == -1) return st;
        }
        else {
            if (s[t[st.v].l + st.pos]
                != s[l])
                return state (-1, -1);
```

```

        if (r-1 < t[st.v].len() -
            st.pos)
            return state (st.v, st.
                pos + r-1);
        l += t[st.v].len() - st.pos
        ;
        st.pos = t[st.v].len();
    }
    return st;
}

int split (state st) {
    if (st.pos == t[st.v].len())
        return st.v;
    if (st.pos == 0)
        return t[st.v].par;
    node v = t[st.v];
    int id = sz++;
    t[id] = node (v.l, v.l+st.pos, v.
        par);
    t[v.par].get( s[v.l] ) = id;
    t[id].get( s[v.l+st.pos] ) = st.v;
    t[st.v].par = id;
    t[st.v].l += st.pos;
    return id;
}

int get_link (int v) {
    if (t[v].link != -1) return t[v].
        link;
    if (t[v].par == -1) return 0;
    int to = get_link (t[v].par);
    return t[v].link = split (go (state
        (to, t[to].len()), t[v].l + (t[v]
        .par==0), t[v].r));
}

void tree_extend (int pos) {
    for (;;) {
        state nptr = go (ptr, pos, pos
            +1);
        if (nptr.v != -1) {
            ptr = nptr;
            return;
        }

        int mid = split (ptr);
        int leaf = sz++;
        t[leaf] = node (pos, n, mid);
        t[mid].get( s[pos] ) = leaf;

        ptr.v = get_link (mid);
        ptr.pos = t[ptr.v].len();
        if (!mid) break;
    }
}

void build_tree() {
    sz = 1;
    for (int i=0; i<n; ++i)
        tree_extend (i);
}

```

}

Tudo comprimido:

```

const int N=1000000, // maximum
    possible number of nodes in suffix
    tree
    INF=10000000000; // infinity
    constant
string a; // input string for
    which the suffix tree is being built
int t[N][26], // array of transitions
    (state, letter)
l[N], // left...
r[N], // ...and right boundaries
    of the substring of a which
    correspond to incoming edge
p[N], // parent of the node
s[N], // suffix link
tv, // the node of the current
    suffix (if we're mid-edge, the
    lower node of the edge)
tp, // position in the string
    which corresponds to the
    position on the edge (between l[
    tv] and r[tv], inclusive)
ts, // the number of nodes
la; // the current character in
    the string

void ukkadd(int c) { // add character s
    to the tree
    suff;; // we'll return here
    after each transition to the
    suffix (and will add character
    again)
    if (r[tv]<tp) { // check whether we
        're still within the boundaries
        of the current edge
        // if we're not, find the next
        edge. If it doesn't exist,
        create a leaf and add it to
        the tree
        if (t[tv][c]==-1) {t[tv][c]=ts;
            l[ts]=la;p[ts++]=tv;tv=s[tv
            ];tp=r[tv]+1;goto suff;}
        tv=t[tv][c];tp=l[tv];
    } // otherwise just proceed to the
    next edge
    if (tp==-1 || c==a[tp]-'a')
        tp++; // if the letter on the
        edge equal c, go down that
        edge
    else {
        // otherwise split the edge in
        two with middle in node ts
        l[ts]=l[tv];r[ts]=tp-1;p[ts]=p[
        tv];t[ts][a[tp]-'a']=tv;
        // add leaf ts+1. It
        corresponds to transition
        through c.
        t[ts][c]=ts+1;l[ts+1]=la;p[ts

```



```

        +1]=ts;
    // update info for the current
    // node – remember to mark ts
    // as parent of tv
    l[tv]=tp; p[tv]=ts; t[p[ts]][a[l[
    ts]]-'a']=ts; ts+=2;
    // prepare for descent
    // tp will mark where are we in
    // the current suffix
    tv=s[p[ts-2]]; tp=l[ts-2];
    // while the current suffix is
    // not over, descend
    while (tp<=r[ts-2]) {tv=t[tv][a
    [tp]-'a']; tp+=r[tv]-l[tv
    ]+1;}
    // if we're in a node, add a
    // suffix link to it, otherwise
    // add the link to ts
    // (we'll create ts on next
    // iteration).
    if (tp==r[ts-2]+1) s[ts-2]=tv;
    else s[ts-2]=ts;
    // add tp to the new edge and
    // return to add letter to
    // suffix
    tp=r[tv]-(tp-r[ts-2])+2; goto
    suff;
}

void build() {
    ts=2;
    tv=0;
    tp=0;
    fill(r, r+N, (int)a.size()-1);
    // initialize data for the root of
    // the tree
    s[0]=1;
    l[0]=-1;
    r[0]=-1;
    l[1]=-1;
    r[1]=-1;
    memset(t, -1, sizeof t);
    fill(t[1], t[1]+26, 0);
    // add the text to the tree, letter
    // by letter
    for (la=0; la<(int)a.size(); ++la)
        ukkadd(a[la]-'a');
}

```

4.7 Autômato de sufixos

4.8 Fatoração de Lyndon

Uma string é dita simples (ou uma palavra de Lyndon) se for estritamente menor que qualquer um de seus sufixos não triviais. Pode ser demonstrado que uma string é simples se e só se for estritamente menor que todos seus shifts cíclicos não triviais.

Seja dada uma string s . A fatoração de Lyndon de s é uma fatoração $s = w_1 w_2 \dots w_k$ em que todas as

strings w_i são simples e $w_1 \geq w_2 \geq \dots \geq w_k$. Pode ser demonstrado que para toda string uma fatoração existe e é única.

O algoritmo de Duval constrói a fatoração de Lyndon em tempo $O(n)$ usando $O(1)$ de memória adicional.

```

vector<string> duval(string const& s) {
    int n = s.size();
    int i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.
            substr(i, j - k));
            i += j - k;
        }
    }
    return factorization;
}

```

Problema 4.8.1: Encontrando o menor shift cíclico.

Solução: Seja s a string. Construímos a fatoração de Lyndon da string s em tempo $O(n)$. Então basta pegar uma string simples da fatoração que começa numa posição menor que n e termina numa posição $\geq n$. A posição inicial dessa string simples vai ser o início do menor shift cíclico.

```

string min_cyclic_string(string s) {
    s += s;
    int n = s.size();
    int i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k)
            i += j - k;
    }
    return s.substr(ans, n / 2);
}

```

4.9 Extras

4.9.1 Parsing

Nesse primeiro código, assumimos que os operadores são binários e left-associative. Parêntesis são

permitidos. Temos uma stack para números e outra para operadores e parêntesis.

```
bool delim(char c) {
    return c == ' ';
}

bool is_op(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

int priority(char op) {
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    int r = st.top(); st.pop();
    int l = st.top(); st.pop();
    switch (op) {
        case '+': st.push(l + r); break;
        case '-': st.push(l - r); break;
        case '*': st.push(l * r); break;
        case '/': st.push(l / r); break;
    }
}

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    for (int i = 0; i < (int)s.size(); i++) {
        if (delim(s[i]))
            continue;

        if (s[i] == '(') {
            op.push('(');
        } else if (s[i] == ')') {
            while (op.top() != '(') {
                process_op(st, op.top());
                op.pop();
            }
            op.pop();
        } else if (is_op(s[i])) {
            char cur_op = s[i];
            while (!op.empty() &&
                priority(op.top()) >=
                priority(cur_op)) {
                process_op(st, op.top());
                op.pop();
            }
        }
    }
}
```

```

    }
    op.push(cur_op);
} else {
    int number = 0;
    while (i < (int)s.size() &&
        isalnum(s[i]))
        number = number * 10 +
            s[i++] - '0';
    --i;
    st.push(number);
}
}

while (!op.empty()) {
    process_op(st, op.top());
    op.pop();
}
return st.top();
}
```

Caso tenhamos associatividade pela direita por ex $a**b**c$ dever ser percebido como $a**(b**c)$ então precisamos substituir a linha

```
while (!op.empty() && priority(op.top())
    ) >= priority(cur_op))
```

por

```
while (!op.empty() && (
    (left_assoc(cur_op) && priority
        (op.top()) >= priority(
            cur_op)) ||
    (!left_assoc(cur_op) &&
        priority(op.top()) >
            priority(cur_op))
    ))
```

Também pode ser que existam unary operators. Por exemplo o unary plus e unary minus (nesse caso precisamos decidir se é binário ou unário). Note também que alguns unary operators são right associative. Aqui uma implementação para os 4 binary operators e unary operators + e -:

```
bool delim(char c) {
    return c == ' ';
}

bool is_op(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

bool is_unary(char c) {
    return c == '+' || c == '-';
}

int priority(char op) {
    if (op < 0) // unary operator
        return 3;
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
}
```

```

        return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    if (op < 0) {
        int l = st.top(); st.pop();
        switch (-op) {
            case '+': st.push(l); break;
            case '-': st.push(-l); break;
        }
    } else {
        int r = st.top(); st.pop();
        int l = st.top(); st.pop();
        switch (op) {
            case '+': st.push(l + r); break;
            case '-': st.push(l - r); break;
            case '*': st.push(l * r); break;
            case '/': st.push(l / r); break;
        }
    }
}

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    bool may_be_unary = true;
    for (int i = 0; i < (int)s.size(); i++) {
        if (delim(s[i]))
            continue;

        if (s[i] == '(') {
            op.push('(');
            may_be_unary = true;
        } else if (s[i] == ')') {
            while (op.top() != '(') {
                process_op(st, op.top());
                op.pop();
            }
            op.pop();
            may_be_unary = false;
        } else if (is_op(s[i])) {
            char cur_op = s[i];
            if (may_be_unary && is_unary(cur_op))
                cur_op = -cur_op;
            while (!op.empty() && (cur_op >= 0 &&
                priority(op.top()) >= priority(
                    cur_op)) ||
                (cur_op < 0 &&

```

```

                priority(op.top()) >= priority(
                    cur_op))
            ) {
                process_op(st, op.top());
                op.pop();
            }
            op.push(cur_op);
            may_be_unary = true;
        } else {
            int number = 0;
            while (i < (int)s.size() &&
                isalnum(s[i]))
                number = number * 10 +
                    s[i++] - '0';
            --i;
            st.push(number);
            may_be_unary = false;
        }
    }

    while (!op.empty()) {
        process_op(st, op.top());
        op.pop();
    }
    return st.top();
}

```

4.9.2 Algoritmo de Manacher

Dada uma string s de tamanho n , queremos achar todos os pares (i, j) tais que $s[i \dots j]$ é um palíndromo. A princípio isso pode parecer impossível, já que é possível que existam $O(n^2)$ substrings palindrômicas. Entretanto, se nós mantivermos eles de maneira compacta, isso é resolvido: para cada posição i , encontramos o número de palíndromos não vazios centrados nessa posição. Isso se aproveita do fato de que se temos um palíndromo de tamanho l centrado em i também temos de tamanhos $l - 2, l - 4$, etc.. Para palíndromos pares assumimos que estão centrados em $s[i]$ e $s[i - 1]$. Trabalharemos, então, com as arrays $d_{\text{odd}}[i]$ e $d_{\text{even}}[i]$. Por exemplo, se $s = abababc$, $d_{\text{odd}}[3] = 3$. Se $s = cbaabd$, $d_{\text{even}}[3] = 2$.

Dá pra resolver com string hashing em $O(n \log n)$ e com árvores de sufixos e fast LCA em $O(n)$, mas esse método daqui é mais simples e tem menos constante escondida.

Aqui a implementação para palíndromos de tamanho ímpar:

```

vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 1, r = 1;
    for (int i = 1; i <= n; i++) {
        p[i] = max(0, min(r - i, p[l +
            (r - i)]));
    }
}

```

```

    while (s[i - p[i]] == s[i + p[i]]) {
        p[i]++;
    }
    if (i + p[i] > r) {
        l = i - p[i], r = i + p[i];
    }
}
return vector<int>(begin(p) + 1,
    end(p) - 1);
}

```

A implementação para o caso par é mais difícil e é bem fácil errar alguma coisinha. O que podemos fazer é reduzir o problema todo pra o caso em que temos que lidar com palíndromos de tamanho ímpar. Para tanto, adicionamos símbolos # entre as letras e no final e no início da string. Com isso, teremos um único array d . Em particular, se $s = abcba$, estaremos lidando com a string $\#a\#b\#c\#b\#a\#\#$ e a partir dela obtaremos uma array $d = [1, 2, 1, 2, 1, 4, 1, 8, 1, 4, 1, 2, 1, 2, 1]$. A partir dela, teremos que $d[2i] = 2d_{\text{even}}[i] + 1$ e $d[2i + 1] = 2d_{\text{odd}}[i]$. Sendo mais preciso, podemos nos aproveitar da implementação anterior e utilizar a seguinte:

```

vector<int> manacher(string s) {
    string t;
    for (auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1,
        end(res) - 1);
}

```

4.9.3 Encontrando repetições

Uma repetição é duas ocorrências de uma string consecutivas, isto é, é um par (i, j) tal que $s[i \dots j]$ consiste de duas strings concatenadas. O desafio é encontrar todas as repetições de uma dada string. Também podemos nos interessar por encontrar qualquer repetição ou encontrar a repetição mais longa.

Podem existir $O(n^2)$ repetições, mas isso não impede de calcular o número delas em $O(n \log n)$, pois o algoritmo dá às repetições uma forma comprimida em grupos.

Alguns resultados interessantes acerca do número de repetições:

1. O número de repetições primitivas (aquelas nas quais as metades não são repetições) é, no máximo, $O(n \log n)$
2. Se encodarmos repetições como tuplas de números (i, p, r) , em que i é a posição de início, p é o tamanho da string que se repete e r o número de repetições, então todas as repetições podem ser descritas com uma quantidade $O(n \log n)$ dessas triplas.

3. Strings de fibonacci definidas por $t_0 = a, t_1 = b, t_i = t_{i-1} + t_{i-2}$ são fortemente periódicas. O número de repetições na string de fibonacci f_i é $O(f_n \log f_n)$.

Algoritmo de Main-Lorentz: Encontra todas as tuplas de tamanho quatro $(cntr, l, k_1, k_2)$, em que $cntr$ é o último índice da primeira string da repetição em questão, l é o tamanho da repetição e k_1 e k_2 são relacionadas ao funcionamento interno do algoritmo.

Se você quiser só encontrar o número de repetições na string ou só quer achar a maior repetição da string, essa informação sobre as tuplas vai ser suficiente e o runtime ainda será $O(n \log n)$.

Note que se você expandir as tuplas para pegar as posições inicial e final de cada repetição, então o runtime será $O(n^2)$ (afinal, podem existir $O(n^2)$ repetições). Nessa implementação, guardamos todas as repetições encontradas num vector de pair e pegamos os pairs de início e fim.

```

vector<int> z_function(string const& s)
{
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

int get_z(vector<int> const& z, int i)
{
    if (0 <= i && i < (int)z.size())
        return z[i];
    else
        return 0;
}

vector<pair<int, int>> repetitions;

void convert_to_repetitions(int shift,
    bool left, int cntr, int l, int k1,
    int k2) {
    for (int l1 = max(1, l - k2); l1 <= min(l, k1); l1++) {
        if (left && l1 == 1) break;
        int l2 = l - l1;
        int pos = shift + (left ? cntr - l1 : cntr - l - l1 + 1);
        repetitions.emplace_back(pos, pos + 2 * l1 - 1);
    }
}

```

```

}

void find_repetitions(string s, int
    shift = 0) {
    int n = s.size();
    if (n == 1)
        return;

    int nu = n / 2;
    int nv = n - nu;
    string u = s.substr(0, nu);
    string v = s.substr(nu);
    string ru(u.rbegin(), u.rend());
    string rv(v.rbegin(), v.rend());

    find_repetitions(u, shift);
    find_repetitions(v, shift + nu);

    vector<int> z1 = z_function(ru);
    vector<int> z2 = z_function(v + '#'
        + u);
    vector<int> z3 = z_function(ru + '#'
        + rv);
    vector<int> z4 = z_function(v);

    for (int cntr = 0; cntr < n; cntr
        ++){
        int l, k1, k2;
        if (cntr < nu) {
            l = nu - cntr;
            k1 = get_z(z1, nu - cntr);
            k2 = get_z(z2, nv + 1 +
                cntr);
        } else {
            l = cntr - nu + 1;
            k1 = get_z(z3, nu + 1 + nv
                - 1 - (cntr - nu));
            k2 = get_z(z4, (cntr - nu)
                + 1);
        }
        if (k1 + k2 >= l)
            convert_to_repetitions(
                shift, cntr < nu, cntr,
                l, k1, k2);
    }
}

```

5 Álgebra e Teoria dos Números

5.1 Exponenciação binária

```
long long binpow(long long a, long long
    b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

Aplicações: Calcular $x^n \bmod m$, usar exponenciação de matrizes para calcular F_n , aplicar uma permutação k vezes, aplicar uma transformação linear num conjunto de pontos, tamanho de caminhos de tamanho k num grafo (elevando a matriz de adjacências), etc..

5.2 MDC e MMC

Ambos em $O(\log(\min(a, b)))$

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}

int iterative_gcd(int a, int b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}

// Nao diminui complexidade mas acelera
// msm assim
// Nunca vi um problema que precisasse
// disso
int fast_gcd(int a, int b) {
    if (!a || !b)
        return a | b;
    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    b >>= __builtin_ctz(b);
    if (a > b)
        swap(a, b);
    b -= a;
    while (b) {
        return a << shift;
    }
}
```

5.3 Algoritmo de Euclides Estendido

Dados $a, b \in \mathbb{Z}$, acha $x, y \in \mathbb{Z}$ tais que

$$ax + by = \gcd(a, b)$$

em $O(\log(\min(a, b)))$. Também produz resultados corretos para negativos.

Os x e y achados por esse algoritmo são ambos os menores possíveis (em módulo). Em particular, você tem a garantia de que $|x| \leq b/2$ e $|y| \leq a/2$. Além disso, não tem risco de overflow no meio dele.

Esse algoritmo deu AC num problema que pedia x e y com $|x| + |y|$ mínimo e $x \leq y$ (se tivesse empate) com $a, b \leq 10^9 + 1$.

```
int iterative_gcd(int a, int b, int& x,
    int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x -
            q * x1);
        tie(y, y1) = make_tuple(y1, y -
            q * y1);
        tie(a1, b1) = make_tuple(b1, a1
            - q * b1);
    }
    return a1;
}

int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

Problema 5.3.1: Dados $a, m \in \mathbb{Z}$ com $\gcd(a, m) = 1$, achar $b \in \mathbb{Z}_m$ tal que $a \cdot b \equiv 1 \pmod{m}$ em $O(\log(\min(a, m)))$

Solução: Aplicar o algoritmo estendido de euclides em a e m de modo a achar x, y tais que $ax + my = 1$. O que você procura é $(x \% m + m) \% m$.

5.4 Equações Diofantinas Lineares

Sejam $a, b, c \in \mathbb{Z}$. Queremos investigar soluções com $x, y \in \mathbb{Z}$ de

$$ax + by = c$$

o caso $a = 0$ ou $b = 0$ é trivial. **Assumiremos**, então, $a, b \neq 0$.

Seja x_0, y_0 uma solução qualquer. As soluções gerais são dadas por $x = x_0 + k \cdot \frac{b}{g}$ e $y = y_0 - k \cdot \frac{a}{g}$.

Problema 5.4.1: Ache qualquer solução

Solução:

```

bool find_any_solution(int a, int b,
    int c, int &x0, int &y0, int &g) {
    g = gcd(abs(a), abs(b), x0, y0); //
    euclides ext
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

```

Problema 5.4.2: Ache todas as soluções para $\min_x \leq x \leq \max_x$ e $\min_y \leq y \leq \max_y$.

Solução:

```

// passei num problema em que x, y, a,
// b, c tinham modulo <= 10^8
// precisava mudar de int pra lli os x,
// os y, o gcd e fazer
// que nem botei no comentario pro
// shift
void shift_solution(int &x, int &y,
    int a, int b, int cnt) {
    x += cnt * b; // 1LL * cnt * b....
    y -= cnt * a;
}

int find_all_solutions(int a, int b,
    int c, int minx, int maxx, int miny,
    int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b, c, x,
        y, g))
        return 0;
    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx -
        x) / b);
    if (x < minx)
        shift_solution(x, y, a, b,
            sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx -
        x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -
            sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny -
        y) / a);

```

```

    if (y < miny)
        shift_solution(x, y, a, b, -
            sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy -
        y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b,
            sign_a);
    int rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}

```

Problema 5.4.3: Quantas soluções não negativas?

Solução:

```

long long count_nonegative_solution(int
    a, int b, int c) {
    return find_all_solutions(1l(a), 1l
        (b), 1l(c), 0LL, 1l(c / a + 1),
        0LL, 1l(c / b + 1));
}

```

Problema 5.4.4: Ache a solução com menor valor de $x + y$ (geralmente é dada uma restrição a mais para que a resposta não seja $-\infty$).

Solução: Basta usar a ideia de que $x + y = x_0 + y_0 + k \cdot \frac{b-a}{g}$.

Problema 5.4.5: Sejam M, N inteiros positivos ($1 \leq M \leq 500$, $1 \leq N \leq 100$) e uma N -tupla de inteiros positivos $(a_1, \dots, a_N)$ ($1 \leq a_i \leq 10$, $1 \leq i \leq N$). Encontre o número de N -tuplas de inteiros positivos $(k_1, \dots, k_N)$ tais que $a_1 k_1 + \dots + a_N k_N = M$.

Solução: TODO

Problema 5.4.6: Qual o menor valor de n tal que $ax + by = N$ admite solução com $x, y \geq 0$ para todo $N \geq n$?

Solução: $n = (a-1)(b-1)$. Para o caso de três variáveis já fica extremamente complicado abordando apenas matematicamente pelo menos.

5.5 Números de fibonacci

$$F_0 = 0, F_1 = 1, F_{n+2} = F_{n+1} + F_n$$

5.5.1 Propriedades interessantes

1. Identidade de Cassini

$$F_{n-1} \cdot F_{n+1} - F_n^2 = (-1)^n$$

2. Regra da adição

$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$$

3. Aplicando $k = n$:

$$F_{2n} = F_n(F_{n+1} + F_{n-1})$$

4. Disso, prova-se que $F_n | F_{nk}$ 5. Se $F_n | F_m$, então $n | m$

6. Identidade do GCD

$$\gcd(F_m, F_n) = F_{\gcd(n, m)}$$

7. São os números que tornam menos eficiente o algoritmo de euclides

Se $n = 2^k$, então $\pi(n) = \frac{3n}{2}$. Se $n = 5^k$, então $\pi(n) = 4n$.

$\pi(n)$	+1	+2	+3	+4	+5	+6	+7	+8	+9	+10	+11	+12
0+	1	3	8	6	20	24	16	12	24	60	10	24
12+	28	48	40	24	36	24	18	60	16	30	48	24
24+	100	84	72	48	14	120	30	48	40	36	80	24
36+	76	18	56	60	40	48	88	30	120	48	32	24
48+	112	300	72	84	108	72	20	48	72	42	58	120
60+	60	30	48	96	140	120	136	36	48	240	70	24
72+	148	228	200	18	80	168	78	120	216	120	168	48
84+	180	264	56	60	44	120	112	48	120	96	180	48
96+	196	336	120	300	50	72	208	84	80	108	72	72
108+	108	60	152	48	76	72	240	42	168	174	144	120
120+	110	60	40	30	500	48	256	192	88	420	130	120
132+	144	408	360	36	276	48	46	240	32	210	140	24

5.5.2 Fibonacci Coding

De acordo com o teorema de Zeckendorf, todo número natural pode ser representado de maneira única como soma de números de fibonacci não consecutivos: $N = F_{k_1} + F_{k_2} + \dots + F_{k_r}$ com $k_1 \geq k_2 + 2, \dots, k_r \geq 2$. Então dá pra escrever o código como $d_0 d_1 \dots d_s 1$, em que $d_i = 1$ se F_{i+2} é usado e um 1 indica o término do código.

Por exemplo, $1 = F_2 = (11)_F$, $2 = F_3 = (011)_F$, $9 = (100011)_F$

Dá pra usar o seguinte algoritmo:

1. Itere até achar o maior menor ou igual a n
2. Subtraia-o de n e coloque o 1 correspondente
3. Repita até o 0
4. Adicione o 1 para indicar o final

Para decodificar é trivial.

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}$$

Pondo $n = 2k$, obtemos as equações $F_{2k+1} = F_k^2 + F_{k+1}^2$ e $F_{2k} = F_k(2F_{k+1} - F_k)$. A partir delas também dá pra calcular F_n .

5.5.3 Periodicidade

Vamos analisar a sequência módulo m . Ela é periódica. O seu período é o pisano period $\pi(n)$. É desconhecido se $\pi(p^k) = p^{k-1}\pi(p)$ para todo primo p e k inteiro (então podemos assumir que isso sempre vale hahaha. Ou passamos o problema ou desprovamos uma conjectura muito conhecida. Os dois outcomes são bons).

Uma consequência do teorema chinês dos restos é que $\pi(p_1^{a_1} \dots p_k^{a_k}) = \pi(p_1^{a_1}) \dots \pi(p_k^{a_k})$

Seja p primo. Se módulo 10 $p \equiv \pm 1$, então $\pi(p)|p-1$. Se $p \equiv \pm 3$, então $\pi(p)$ é o quociente entre $2(p+1)$ e um divisor ímpar.

É sabido que $\pi(n) \leq 6n$ com igualdade se, e só se, $n = 2 \cdot 5^r$ com $r \geq 1$.

5.5.4 Problemas

Problema 5.5.1: Determine $F_0 + F_1 + \dots + F_n$

Solução: $F_{n+2} - 1$

Problema 5.5.2: Dado um certo resto f por $m = 10^{13}$, qual o primeiro n tal que $F_n \equiv r$?

Solução: A ideia é principal é baby-step-giant-step. Sejam $a, b | m$ com $\gcd(a, b) = 1$. Seja F o fibonacci tal que $F \equiv f \pmod{m}$. Então, $F \equiv f \pmod{a}$ e $F \equiv f \pmod{b}$. Encontre todas as ocorrências de $f \pmod{a}$ na sequência de fibonacci módulo a . Faça o mesmo pro b . Suponha que na posição i da primeira, temos algo que satisfaça e que na posição j da segunda também. Seja $t(m)$ o período da sequência de fibonacci módulo m . Temos $t(ab) = \text{lcm}(t(a), t(b))$. Então resolvendo a equação diofantina $i + t(a) \cdot x = j + t(b) \cdot y$, achamos uma posição em ab que satisfaz o que queremos. Então é só bruteforçar o k em $t(ab) \cdot k + u$. (pra pular $t(ab)$ posições é só multiplicar por uma matriz). No caso de $m = 10^{13}$ dá pra tomar $a = 5^9$ e $b = 2^{13}$.

5.6 Primos

5.6.1 Crivo de Eratóstenes

Acha todos os primos de 1 até n usando $O(n)$ memória e $O(n \log \log n)$ tempo

```
int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i * i <= n; i++) {
    if (is_prime[i]) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}
```

É possível fazer o crivo usando apenas $O(\sqrt{n} + S)$ de memória utilizando divisão em blocos de tamanho S . Use $10^4 \leq S \leq 10^5$

```

int count_primes(int n) {
    //conta o numero de primos de 1
    ateh n
    const int S = 10000;

    vector<int> primes;
    int nsqrt = sqrt(n);
    vector<char> is_prime(nsqrt + 2,
        true);
    for (int i = 2; i <= nsqrt; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (int j = i * i; j <=
                nsqrt; j += i)
                is_prime[j] = false;
        }
    }

    int result = 0;
    vector<char> block(S);
    for (int k = 0; k * S <= n; k++) {
        fill(block.begin(), block.end(),
            true);
        int start = k * S;
        for (int p : primes) {
            int start_idx = (start + p
                - 1) / p;
            int j = max(start_idx, p) *
                p - start;
            for (; j < S; j += p)
                block[j] = false;
        }
        if (k == 0)
            block[0] = block[1] = false;
        for (int i = 0; i < S && start
            + i <= n; i++) {
            if (block[i])
                result++;
        }
    }
    return result;
}

```

Como encontrar os números num intervalo pequeno de tamanho 10^7 $[L, R]$ mas com $R \leq 10^{12}$? Encontre os primos até $\sqrt{R}$ e marque todos os compostos no segmento $[L, R]$. Isso roda em $O((R - L + 1) \log \log(R) + \sqrt{R} \log \log \sqrt{R})$.

```

vector<char> segmentedSieve(long long L
    , long long R) {
    // generate all primes up to sqrt(R)
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j

```

```

                <= lim; j += i)
                mark[j] = true;
        }
    }

    vector<char> isPrime(R - L + 1,
        true);
    for (long long i : primes)
        for (long long j = max(i * i, (
            L + i - 1) / i * i); j <= R;
            j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    return isPrime;
}

```

Na verdade uma versão disso em $O((R - L + 1) \log(R) + \sqrt{R})$ não exige a pre-geração dos primos:

```

vector<char> segmentedSieveNoPreGen(
    long long L, long long R) {
    vector<char> isPrime(R - L + 1,
        true);
    long long lim = sqrt(R);
    for (long long i = 2; i <= lim; ++i)
        for (long long j = max(i * i, (
            L + i - 1) / i * i); j <= R;
            j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    return isPrime;
}

```

Outra versão do crivo é em $O(n)$. O problema é que usa $32\times$ a memória (então só faz sentido pra $n \leq 10^7$). E também não é tão "mais rápido" que o crivo normal. É até mais lento que o crivo em blocos.

O benefício de usar ele é que com ele dá pra calcular facilmente a fatoração de um número, pois a array $lp[i]$ calcula o menor fator primo de todos os números no range $[2; n]$.

```

const int N = 100000000;
vector<int> lp(N+1);
vector<int> pr;

for (int i=2; i <= N; ++i) {
    if (lp[i] == 0) {
        lp[i] = i;
        pr.push_back(i);
    }
    for (int j = 0; i * pr[j] <= N; ++j) {
        lp[i * pr[j]] = pr[j];
        if (pr[j] == lp[i]) {
            break;
        }
    }
}

```

Crivo mais poderoso que já vi. Ele funcionou num problema que pedia a soma dos divisores de números $\leq 10^{16}$ (olhar como resolvi na seção de funções multiplicativas) que precisou dos primos de 1 até 10^8 .

```
#define MAX 1000000000
#define LMT 10000

int flag[MAX / 64];
int primes[5761460], total;

#define chkC(n) (flag[n >> 6] & (1 << ((n >> 1) & 31)))
#define setC(n) (flag[n >> 6] |= (1 << ((n >> 1) & 31)))

void sieve() {
    int i, j, k;
    flag[0] = 0;
    for (i = 3; i < LMT; i += 2)
        if (!chkC(i))
            for (j = i * i, k = i << 1; j < MAX; j += k)
                setC(j);
    primes[(j = 0)++] = 2;
    for (i = 3; i < MAX; i += 2)
        if (!chkC(i)) primes[j++] = i;
    total = j;
}
```

5.6.2 Testes de primalidade

Clássico $O(\sqrt{n})$

```
bool isPrime(int x) {
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0)
            return false;
    }
    return x >= 2;
}
```

MillerRabin: o determinístico passa tranquilo pra $t \leq 500$ casos e $n \leq 2^{63} - 1$

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpower(u64 base, u64 e, u64 mod)
{
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (u128)result *
                    base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}
```

```
bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};
```

// NAO DETERMINISTICO

```
bool MillerRabin(u64 n, int iter=5) {
    // returns true if n is probably
    // prime, else returns false.
    if (n < 4)
        return n == 2 || n == 3;

    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s++;
    }

    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s))
            return false;
    }
    return true;
}
```

// DETERMINISTICO

```
bool MillerRabin(u64 n) { // returns
    true if n is prime, else returns
    false.
    if (n < 2)
        return false;

    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}
```

5.6.3 Fatoração

Para $n \leq 10^{12}$ dá pra usar um crivo até 10^6 e gerar uma lista de primos e ir testando.

Para $n \leq 10^{18}$, usar pollard

```
typedef long long unsigned int llui;
typedef long long int lli;
typedef long double float64;

llui mul_mod(llui a, llui b, llui m) {
    llui y = (llui)((float64)a * (
        float64)b / m + (float64)1 / 2);
    y = y * m;
    llui x = a * b;
    llui r = x - y;
    if ((lli)r < 0) {
        r = r + m;
        y = y - 1;
    }
    return r;
}

llui C, a, b;
llui gcd() {
    llui c;
    if (a > b) {
        c = a;
        a = b;
        b = c;
    }
    while (1) {
        if (a == 1LL) return 1LL;
        if (a == 0 || a == b) return b;
        c = a;
        a = b % a;
        b = c;
    }
}

llui f(llui a, llui b) {
    llui tmp;
    tmp = mul_mod(a, a, b);
    tmp += C;
    tmp %= b;
    return tmp;
}

llui pollard(llui n) {
    if (!(n & 1)) return 2;
    C = 0;
    llui iteracoes = 0;
    while (iteracoes <= 1000) {
        llui x, y, d;
        x = y = 2;
        d = 1;
        while (d == 1) {
            x = f(x, n);
            y = f(f(y, n), n);
            llui m = (x > y) ? (x - y)
                : (y - x);
            a = m;
```

```
        b = n;
        d = gcd();
    }
    if (d != n) return d;
    iteracoes++;
    C = rand();
}

return 1;
}

llui pot(llui a, llui b, llui c) {
    if (b == 0) return 1;
    if (b == 1) return a % c;
    llui resp = pot(a, b >> 1, c);
    resp = mul_mod(resp, resp, c);
    if (b & 1) resp = mul_mod(resp, a,
        c);
    return resp;
}

// Rabin–Miller primality testing
algorithm
bool isPrime(llui n) {
    llui d = n - 1;
    llui s = 0;
    if (n <= 3 || n == 5) return true;
    if (!(n & 1)) return false;
    while (!(d & 1)) {
        s++;
        d >>= 1;
    }
    for (llui i = 0; i < 32; i++) {
        llui a = rand();
        a <<= 32;
        a += rand();
        a %= (n - 3);
        a += 2;
        llui x = pot(a, d, n);
        if (x == 1 || x == n - 1)
            continue;
        for (llui j = 1; j <= s - 1; j
            ++){
            x = mul_mod(x, x, n);
            if (x == 1) return false;
            if (x == n - 1) break;
        }
        if (x != n - 1) return false;
    }
    return true;
}

map<llui, int> factors;
// Precondition: factors is an empty
// map, n is a positive integer
// Postcondition: factors[p] is the
// exponent of p in prime factorization
// of n
void fact(llui n) {
    if (!isPrime(n)) {
        llui fac = pollard(n);
        fact(n / fac);
        fact(fac);
    }
```

```

    } else {
        map<llui, int>::iterator it;
        it = factors.find(n);
        if (it != factors.end()) {
            (*it).second++;
        } else {
            factors[n] = 1;
        }
    }
}

```

5.7 Funções comuns de teoria dos números

5.7.1 Função totiente de Euler

Seja $\phi(n)$ a função totiente de euler.

Código para calcular $\phi(n)$ em $O(\sqrt{n})$:

```

int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}

```

Código para calcular $\phi(n)$ de 1 a n em $O(n \log \log n)$ com crivo:

```

void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}

```

Fatos conhecidos:

$$\phi(p_1^{\alpha_1} \dots p_k^{\alpha_k}) = p_1^{\alpha_1} \dots p_k^{\alpha_k} \left(1 - \frac{1}{p_1}\right) \dots \left(1 - \frac{1}{p_k}\right)$$

$$\sum_{d|n} \phi(d) = n$$

Para calcular $x^n \bmod m$ rápido (mesmo que $\gcd(x, m) \neq 1$) podemos usar:

$$x^n \equiv x^{\phi(m) + [n \bmod \phi(m)]} \pmod{m}$$

5.7.2 Funções quantidade e soma de divisores

Seja $n = p_1^{e_1} \dots p_k^{e_k}$. Então

$$d(n) = \prod_{i=1}^k (e_i + 1)$$

$$\sigma(n) = \prod_{i=1}^k \frac{p_i^{e_i+1} - 1}{p_i - 1}$$

É possível calculá-las em $O(\sqrt{n})$ ingenuamente. Mas se você usar o crivo super-poderoso e ir subindo no vetor de primos, dá até pra calcular a soma dos divisores de vários $n \leq 10^{16}$. (dá pra usar crivo normal também, ele só não vai chegar a 10^{16}).

5.7.3 Função de mobius

$\mu(n) = 0$ se $p^2 | n$ para algum p . Caso contrário, $\mu(n) = (-1)^k$ em que k é o número de fatores primos distintos de n . Por exemplo, $\mu(1) = 1$, $\mu(2) = -1$, $\mu(3) = -1$, $\mu(4) = 0$, $\mu(6) = 1$. Calcula com crivo:

```

const int MAXN = 2e6 + 100;

std::vector<int> prime;
bool is_composite[MAXN];
int mu[MAXN];

void sieve(int n) {
    std::fill(is_composite, is_composite + n, false);
    mu[1] = 1;
    for (int i = 2; i < n; ++i) {
        if (!is_composite[i]) {
            prime.push_back(i);
            mu[i] = -1; // i is prime
        }
        for (int j = 0; j < prime.size() && i * prime[j] < n; ++j) {
            is_composite[i * prime[j]] = true;
            if (i % prime[j] == 0) {
                mu[i * prime[j]] = 0;
                // prime[j] divides i
                break;
            } else {
                mu[i * prime[j]] = -mu[i];
                // prime[j] does not divide i
            }
        }
    }
}

```

5.7.4 Funções aritméticas multiplicativas

Funções nos inteiros tais que $f(ab) = f(a)f(b)$ quando $\gcd(a, b) = 1$. Na grande maioria dos casos,

é fácil adaptar o crivo acima. O único problema é caso fique difícil de fazer o caso em que o primo divide i . O que você pode fazer é criar uma array cnt indicando o expoente do menor fator primo. Aqui um exemplo em que $f(p^k) = k$:

```
std::vector<int> prime;
bool is_composite[MAXN];
int func[MAXN], cnt[MAXN];

void sieve (int n) {
    std::fill (is_composite,
                is_composite + n, false);
    func[1] = 1;
    for (int i = 2; i < n; ++i) {
        if (!is_composite[i]) {
            prime.push_back (i);
            func[i] = 1; cnt[i] = 1;
        }
        for (int j = 0; j < prime.size ()
            && i * prime[j] < n; ++j) {
            is_composite[i * prime[j]] =
                true;
            if (i % prime[j] == 0) {
                func[i * prime[j]] = func[i]
                    / cnt[i] * (cnt[i] +
                        1);
                cnt[i * prime[j]] = cnt[i]
                    + 1;
                break;
            } else {
                func[i * prime[j]] = func[i]
                    * func[prime[j]];
                cnt[i * prime[j]] = 1;
            }
        }
    }
}
```

```
// acha inversos de a[1], a[2], ..., a[
n] mod p primo
// acha em O(n + log p) tempo. Requer O
(n) memo

#include "bits/stdc++.h"
using namespace std;

const long long P = 1000000007;
long long qpow(long long a, long long b
) {
    long long ans = 1;
    while (b) {
        if (b & 1) ans = ans * a % P;
        a = a * a % P;
        b >>= 1;
    }
    return ans;
}

long long n, a[1000010], pre[1000010],
suf[1000010], pr = 1; // prefix,
suffix, product
int main() {
    cin >> n; pre[0] = suf[n + 1] = 1;
    for (int i = 1; i <= n; i++) cin >>
        a[i];
    for (int i = 1; i <= n; i++) {
        pre[i] = pre[i - 1] * a[i] % P;
        suf[n + 1 - i] = suf[n + 2 - i] * a
            [n + 1 - i] % P;
        pr = pr * a[i] % P;
    }
    pr = qpow(pr, P - 2);
    for (int i = 1; i <= n; i++) {
        cout << (pre[i - 1] * suf[i + 1] %
            P) * pr % P << endl;
    }
}
```

5.8 Inversos modulares

Para achar o inverso multiplicativo de a módulo m , uma condição necessária pra ele existir é que $\gcd(a, m) = 1$. E como já falei em 5.3.1, basta usar o algoritmo estendido de euclides.

Se não quiser usar ele, dá pra usar exponenciação binária pra calcular $a^{\phi(m)-1}$. Em particular, se m for primo dá pra fazer isso em a^{m-2} .

Problema 5.8.1: Ache inversos módulo p primo.

Solução 1:

```
// acha inversos de 1 a n mod p primo
// acha em O(n) tempo. Requer O(n) memo

int n = 10, p = 1000000007;
int inv[n + 1];
inv[1] = 1;
for (int i = 2; i <= n; i++) inv[i] =
    1LL * (p - p / i) * inv[p % i] % p;
```

Solução 2:

5.9 Congruências modulares

Sejam a, b e n inteiros dados. Queremos achar os x inteiros em $[0, n - 1]$ tais que

$$a \cdot x \equiv b \pmod{n}$$

Seja $g = \gcd(a, n)$. Seja $a = a'g$, $n = n'g$. Se g não dividir b , não existem soluções. Seja $b = gb'$. A equação original implica

$$a' \cdot x \equiv b' \pmod{n'}$$

Como $\gcd(a', n') = 1$, então só existe uma solução. Então, ao todo, existem g soluções dadas por

$$x_i \equiv x' + i \cdot n' \pmod{n}, i = 0 \dots g - 1$$

Também é possível resolvê-la utilizando o algoritmo estendido de euclides com a equação

$$a \cdot x + n \cdot k = b$$

em que x e k são desconhecidos.

5.9.1 Teorema Chinês dos Restos

Sejam $m_1, \dots, m_k$ coprimos. Sejam a_i inteiros. Queremos achar a tal que $a \equiv a_i \pmod{m_i}$ para todo $1 \leq i \leq k$.

A solução é única mod $m_1 m_2 \dots m_k$.

Para construí-la, seja $M_i = \prod_{j \neq i} m_j$ e $N_i = M_i^{-1} \pmod{m_i}$. Então, módulo $m_1 \dots m_k$, a solução é

$$a \equiv \sum_{i=1}^k a_i M_i N_i$$

```
struct Congruence {
    long long a, m;
};

long long chinese_remainder_theorem(
    vector<Congruence> const&
    congruences) {
    long long M = 1;
    for (auto const& congruence :
        congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence :
        congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i,
            congruence.m);
        solution = (solution + a_i *
            M_i % M * N_i) % M;
    }
    return solution;
}
```

5.9.2 Algoritmo de Garner

É possível usar o teorema chinês dos restos para representar um número muito gigante a utilizando apenas os a_i . Utilizamos a representação

$$a = x_1 + x_2 p_1 + x_3 p_1 p_2 + \dots + x_k p_1 \dots p_{k-1}$$

com $x_i \in [0, p_i)$

O que o algoritmo de Garner faz é computar os dígitos x_i . É possível utilizar esse algoritmo para calcular em $O(k^2)$ os x_i . Eis uma implementação **em java**:

```
final int SZ = 100;
int pr[] = new int[SZ];
int r[][] = new int[SZ][SZ];

void init() {
    for (int x = 1000 * 1000 * 1000, i
        = 0; i < SZ; ++i)
        if (BigInteger.valueOf(x).
            isProbablePrime(100))
```

```
        pr[i++] = x;

    for (int i = 0; i < SZ; ++i)
        for (int j = i + 1; j < SZ; ++j)
            r[i][j] =
                BigInteger.valueOf(pr[i]
                    ).modInverse(
                        BigInteger.valueOf(
                            pr[j]))
                    .intValue();
}

class Number {
    int a[] = new int[SZ];

    public Number() {}

    public Number(int n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n % pr[i];
    }

    public Number(BigInteger n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n.mod(BigInteger.
                valueOf(pr[i])).intValue();
    }

    public Number add(Number n) {
        Number result = new Number();
        for (int i = 0; i < SZ; ++i)
            result.a[i] = (a[i] + n.a[i]
                ) % pr[i];
        return result;
    }

    public Number subtract(Number n) {
        Number result = new Number();
        for (int i = 0; i < SZ; ++i)
            result.a[i] = (a[i] - n.a[i]
                + pr[i]) % pr[i];
        return result;
    }

    public Number multiply(Number n) {
        Number result = new Number();
        for (int i = 0; i < SZ; ++i)
            result.a[i] = ((int)((a[i] *
                11 * n.a[i]) % pr[i]));
        return result;
    }

    public BigInteger bigIntegerValue(
        boolean can_be_negative) {
        BigInteger result = BigInteger.
            ZERO, mult = BigInteger.ONE;
        int x[] = new int[SZ];
        for (int i = 0; i < SZ; ++i) {
            x[i] = a[i];
```



```

        for (int j = 0; j < i; ++j)
        {
            long cur = (x[i] - x[j]
                ) * 11 * r[j][i];
            x[i] = (int)((cur % pr[
                i] + pr[i]) % pr[i]);
        }
        result = result.add(mult.
            multiply(BigInteger.
                valueOf(x[i])));
        mult = mult.multiply(
            BigInteger.valueOf(pr[i]
                ));
    }

    if (can_be_negative)
        if (result.compareTo(mult.
            shiftRight(1)) >= 0)
            result = result.
                subtract(mult);

    return result;
}

```

5.9.3 Fatorial módulo p

Dá pra precalcular em $O(p)$ os fatoriais e em $O(\log_p n)$ o fatorial de um determinado número. Isso usa $O(p)$ de memória.

```

int factmod(int n, int p) {
    vector<int> f(p);
    f[0] = 1;
    for (int i = 1; i < p; i++)
        f[i] = f[i-1] * i % p;

    int res = 1;
    while (n > 1) {
        if ((n/p) % 2)
            res = p - res;
        res = res * f[n%p] % p;
        n /= p;
    }
    return res;
}

```

Truque de mestre! Se $p \approx 10^9$, dá pra hardcodar um vetor fatmod tal que fatmod[i] é $(Si)!$ módulo p em que S é um tamanho de bloco. Dá pra usar $S = 10^6$ pro vetor ter 10^3 caras e isso ser algo colocável no código.

Então ficamos com:

```

const long long int mod = 1e9+7;
const long long int maxn = 1e9;
const int tempo = 1e6;
long long int fatmod[mod/tempo+1] = {
    /* com outro programa calcular os
    numeros e copiar e colar aqui*/};

```

```

long long int computa_fatmod(int n) {
    if (n >= mod) return 0;
    long long int atual = fatmod[n/
        tempo];
    for (int i = (n/tempo)*tempo+1; i
        <= n; i++) {
        atual *= i;
        atual %= mod;
    }
    return atual;
}

```

5.9.4 Fórmula de Legendre

Qual o expoente de p na fatoração prima de $n!$? Sem calcular $n!$, podemos calcular isso em $O(\log_p n)$ com a seguinte fórmula:

$$v_p(n!) = \sum_{i=1}^{\infty} \left\lfloor \frac{n}{p^i} \right\rfloor$$

```

int multiplicity_factorial(int n, int p)
{
    int count = 0;
    do {
        n /= p;
        count += n;
    } while (n);
    return count;
}

```

5.9.5 Logaritmo Discreto

Inteiro x satisfazendo

$$a^x \equiv b \pmod{m}$$

para inteiros a, b, m dados. Será resolvido com o algoritmo **baby-step giant-step** de complexidade $O(\sqrt{m})$. Assumiremos que $0^0 \equiv 1$.

Forma mais simples sem usar unordered map roda em $O(\sqrt{m} \log m)$ (assumiremos nas duas versões $\gcd(a, m) = 1$):

```

int powmod(int a, int b, int m) {
    int res = 1;
    while (b > 0) {
        if (b & 1) {
            res = (res * 111 * a) % m;
        }
        a = (a * 111 * a) % m;
        b >>= 1;
    }
    return res;
}

```

```

int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;
    map<int, int> vals;
    for (int p = 1; p <= n; ++p)
        vals[powmod(a, p * n, m)] = p;
    for (int q = 0; q <= n; ++q) {

```

```

    int cur = (powmod(a, q, m) * 1
    11 * b) % m;
    if (vals.count(cur)) {
        int ans = vals[cur] * n - q
        ;
        return ans;
    }
}
return -1;
}

```

Usando unordered map:

```

// Returns minimum x for which a ^ x %
m = b % m, a and m are coprime.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 111 * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++
    q) {
        vals[cur] = q;
        cur = (cur * 111 * a) % m;
    }

    for (int p = 1, cur = 1; p <= n; ++
    p) {
        cur = (cur * 111 * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur]
            ];
            return ans;
        }
    }
    return -1;
}

```

Caso em que $\gcd(a, m) \neq 1$. Se g não divide b , então não há solução. Se $g|b$, então tomando $a = g\alpha$, $b = g\beta$, $m = g\nu$. Daí, precisamos resolver $\alpha a^{x-1} \equiv \beta \pmod{\nu}$. Mas o nosso algoritmo pode ser facilmente estendido para $ka^x \equiv b$:

```

// Returns minimum x for which a ^ x %
m = b % m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 111 * a / g) % m;
    }

    int n = sqrt(m) + 1;

```

```

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 111 * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++
    q) {
        vals[cur] = q;
        cur = (cur * 111 * a) % m;
    }

    for (int p = 1, cur = k; p <= n; ++
    p) {
        cur = (cur * 111 * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur]
            + add;
            return ans;
        }
    }
    return -1;
}

```

5.9.6 Raízes primitivas

g é dita uma raiz primitiva módulo n se todo número coprimo a n é congruente a uma potência de g módulo n . Se g é uma raiz primitiva e $g^k \equiv a$, então k é dito o índice ou logaritmo discreto de a na base g .

Uma raiz primitiva módulo n existe se e só se:

1. $n = 1, 2, 4$ ou
2. $n = p^k$, p primo ímpar, $k \geq 1$
3. $n = 2p^k$, p primo ímpar, $k \geq 1$

Se existe alguma raiz primitiva módulo n , então existem $\phi(\phi(n))$ delas. O seguinte código acha um gerador em $O(Ans \cdot \log \phi(n) \cdot \log n)$. Sabe-se que $g \in O(\log^6 p)$. Há um detalhe: o código abaixo **só funciona** para p primo. Mas pra funcionar pra qualquer número basta trocar a definição da variável phi:

```

int powmod (int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int (res * 111 * a %
            p), --b;
        else
            a = int (a * 111 * a % p),
            b >>= 1;
    return res;
}

int generator (int p) {
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);

```

```

        while (n % i == 0)
            n /= i;
    }
    if (n > 1)
        fact.push_back(n);

    for (int res=2; res<=p; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size()
            && ok; ++i)
            ok &= powmod(res, phi /
                fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}

```

5.9.7 Raiz discreta

Dado um primo n e dois inteiros a, k , achar todos os x tais que $x^k \equiv a \pmod{n}$.

Solução: achar uma raiz primitiva g módulo n (em $O(\text{Ans} \cdot \log^2 n)$). Depois, se $x \equiv g^y$, basta resolver $(g^k)^y \equiv a$ em y usando o logaritmo discreto.

Problema 5.9.1: Encontrar todas as soluções a partir de uma.

Solução: suponha que conhecemos a solução $x_0 = g^{y_0} \pmod{n}$. Então, todas as soluções são da forma

$$x \equiv g^{y_0 + i \frac{\phi(n)}{\gcd(k, \phi(n))}}$$

com i variando em $\mathbb{Z}$.

O seguinte código mostra todas as soluções:

```

int gcd(int a, int b) {
    return a ? gcd(b % a, a) : b;
}

int powmod(int a, int b, int p) {
    int res = 1;
    while (b > 0) {
        if (b & 1) {
            res = res * a % p;
        }
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

// Finds the primitive root modulo p
int generator(int p) {
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    }
}

```

```

    }
    if (n > 1)
        fact.push_back(n);

    for (int res = 2; res <= p; ++res)
    {
        bool ok = true;
        for (int factor : fact) {
            if (powmod(res, phi /
                factor, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return res;
    }
    return -1;
}

// This program finds all numbers x
such that x^k = a (mod n)
int main() {
    int n, k, a;
    scanf("%d %d %d", &n, &k, &a);
    if (a == 0) {
        puts("1\n0");
        return 0;
    }

    int g = generator(n);

    // Baby-step giant-step discrete
    logarithm algorithm
    int sq = (int) sqrt(n + .0) + 1;
    vector<pair<int, int>> dec(sq);
    for (int i = 1; i <= sq; ++i)
        dec[i-1] = {powmod(g, i * sq *
            k % (n - 1), n), i};
    sort(dec.begin(), dec.end());
    int any_ans = -1;
    for (int i = 0; i < sq; ++i) {
        int my = powmod(g, i * k % (n -
            1), n) * a % n;
        auto it = lower_bound(dec.begin(),
            dec.end(), make_pair(my, 0));
        if (it != dec.end() && it->
            first == my) {
            any_ans = it->second * sq -
                i;
            break;
        }
    }
    if (any_ans == -1) {
        puts("0");
        return 0;
    }

    // Print all possible answers
    int delta = (n-1) / gcd(k, n-1);
    vector<int> ans;
}

```

```

for (int cur = any_ans % delta; cur
    < n-1; cur += delta)
    ans.push_back(powmod(g, cur, n)
    );
sort(ans.begin(), ans.end());
printf("%d\n", ans.size());
for (int answer : ans)
    printf("%d ", answer);
}

```

5.10 Sistemas Numéricos

5.10.1 Ternário balanceado

Usa os dígitos 1, 0 e $Z (= -1)$. Para calcular a partir da base 3, usemos o exemplo $64_{10} = 02101_3$. Você sempre processa da direita pra esquerda. O 1 e 0 são skipados. E também toda vez que encontramos um 2, transformamos ele em Z e adicionamos 1 ao próx dígito (ou seja, existe um carry). Então $1Z101 = 64_{10}$.

5.10.2 Código de Gray

Sistema binário em que valores consecutivos diferem em um bit. Os gray codes para números de 3 bits são 000, 001, 011, 010, 110, 111, 101, 100. Então $G(4) = 110_2 = 6$. É fácil calcular $G(n)$ com operações binárias. Basta usar

```

int g (int n) {
    return n ^ (n >> 1);
}

```

Problema 5.10.1: Dado $g(n)$, recuperar n .

Solução:

```

int rev_g (int g) {
    int n = 0;
    for (; g; g >>= 1)
        n ^= g;
    return n;
}

```

Algumas coisas legais:

1. os códigos de gray de n bits determinam um ciclo hamiltoniano num hiper-cubo de n dimensões
2. dá pra resolver o problema das torres de hanoi. Seja n o número de discos. Comece com $G(0)$ e vá indo de $G(i)$ para $G(i+1)$. Interprete como se o i -ésimo bit do gray code atual representasse o i -ésimo disco. Lembrando que só um bit muda, podemos interpretar a mudança no i -ésimo bit como uma mudança no i -ésimo disco. Note que existe exatamente uma opção de movimento para cada disco em cada etapa. Existem duas opções para o menor disco mas sempre tem a seguinte estratégia. Se n for ímpar, a sequência de menor quantidade de movimentos é $f \rightarrow t \rightarrow r \rightarrow f \rightarrow t \rightarrow r \rightarrow \dots$ em que f é a rod inicial, t é a rod final e r a restante. E se n for ímpar ela é $f \rightarrow r \rightarrow r \rightarrow f \rightarrow r \rightarrow t \rightarrow \dots$.

3. Sendo $G(N)$ a sequência de códigos de Gray para N bits, $G(N)^R$ ela revertida e $aG(N)$ ela concatenada do bit de valor a , então

$$G(N) = 0G(N-1) \cup 1G(N-1)^R$$

Implementação da torre de hanoi (discos de 0 a $n-1$, pilhas de 0 a 2, move tudo da pilha 0 para a 2)

```

int g(int n) { return n ^ (n >> 1); }

void swap_peg(int& x) {
    if (x == 1)
        x = 2;
    else if (x == 2)
        x = 1;
}

int main() {
    int n;
    cin >> n;
    for (int i = 1;; i++) {
        int gi = g(i-1);
        int gip1 = g(i);
        int bitpos = gi ^ gip1;
        int pos_dude = __builtin_ctz(
            bitpos);
        if (pos_dude == n) break;
        int from_peg = (i & (i-1)) % 3;
        int to_peg = ((i | (i-1)) + 1) % 3;
        if (n % 2 == 0) {
            swap_peg(from_peg);
            swap_peg(to_peg);
        }
        cout << "Disco " << pos_dude <<
            " vai de " << from_peg << "
            para " << to_peg << "\n";
    }
}

```

5.11 Manipulação de bits

```

bool isPowerOfTwo(unsigned int n) {
    return n && !(n & (n-1));
}

int countSetBits(int n) {
    int count = 0;
    while (n)
    {
        n = n & (n-1);
        count++;
    }
    return count;
}

// n & (n+1) limpa os trailing ones
// n | (n+1) seta o último bit zero

```

```
// n & (-n) extrai o último bit

// __builtin_popcount(unsigned int)
// numero de bits um

// __builtin_ffs(int) índice do bit 1
// mais a direita

// __builtin_clz(unsigned int) -> qnt
// de leading zeros

// __builtin_ctz(unsigned int) -> qnt
// de trailing zeros

// __builtin_parity() -> paridade do nú
// mero de uns

// NOTA: algumas dessas podem ser
// lentas no GCC se nao ativar um
// target específico de compilador com
// #pragma GCC target("popcnt")
```

5.11.1 Iterar por todas as submáscaras de uma máscara

```
// O(3^n)
for (int s=m; ; s=(s-1)&m) {
    // poe o codigo aqui se quiser
    // considerar o caso s=0
    // se n, bota dps do if
    if (s==0) break;
}
```

5.12 Aritmética de precisão arbitrária

Para representar números muito grandes, podemos usar um vector de int em que cada elemento representa um dígito e isso na base 10^9

```
typedef vector<int> lnum;
const int base = 1000*1000*1000;

// printar
printf ("%d", a.empty() ? 0 : a.back());
;
for (int i=(int)a.size()-2; i>=0; --i)
    printf ("%09d", a[i]);

// leitura (string)
for (int i=(int)s.length(); i>0; i-=9)
    if (i < 9)
        a.push_back (atoi (s.substr (0,
            i).c_str()));
    else
        a.push_back (atoi (s.substr (i
            -9, 9).c_str()));

// leitura (vetor de char)
for (int i=(int)strlen(s); i>0; i-=9) {
    s[i] = 0;
    a.push_back (atoi (i>=9 ? s+i-9 : s
        ));
}
```

```
// se a entrada contiver leading zeros
while (a.size() > 1 && a.back() == 0)
    a.pop_back();

// a+= b
int carry = 0;
for (size_t i=0; i<max(a.size(),b.size()) || carry; ++i) {
    if (i == a.size())
        a.push_back (0);
    a[i] += carry + (i < b.size() ? b[i] : 0);
    carry = a[i] >= base;
    if (carry) a[i] -= base;
}

// a-= b
int carry = 0;
for (size_t i=0; i<b.size() || carry; ++i) {
    a[i] -= carry + (i < b.size() ? b[i] : 0);
    carry = a[i] < 0;
    if (carry) a[i] += base;
}

while (a.size() > 1 && a.back() == 0)
    a.pop_back();

//multiplicacao por b < base
int carry = 0;
for (size_t i=0; i<a.size() || carry; ++i) {
    if (i == a.size())
        a.push_back (0);
    long long cur = carry + a[i] * 111
        * b;
    a[i] = int (cur % base);
    carry = int (cur / base);
}

while (a.size() > 1 && a.back() == 0)
    a.pop_back();

// multiplicacao por outro vector
lnum c (a.size()+b.size());
for (size_t i=0; i<a.size(); ++i)
    for (int j=0, carry=0; j<(int)b.size() || carry; ++j) {
        long long cur = c[i+j] + a[i] *
            111 * (j < (int)b.size() ?
                b[j] : 0) + carry;
        c[i+j] = int (cur % base);
        carry = int (cur / base);
    }
while (c.size() > 1 && c.back() == 0)
    c.pop_back();

// divisao por b < base (a /= b)
int carry = 0;
for (int i=(int)a.size()-1; i>=0; --i)
{
```

```

    long long cur = a[i] + carry * 1ll
        * base;
    a[i] = int (cur / b);
    carry = int (cur % b);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();

```

Também podemos fatorar em fatores primos e trabalhar com eles.

Outra ideia é usar o algoritmo de Garner.

Uma ideia para variáveis de ponto flutuante é guardar um int com o expoente e um double com um valor em $[0.1, 1)$

5.13 FFT

Seja $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ e $B(x)$ definido analogamente. Queremos saber os coeficientes de $A(x) \cdot B(x)$ em $O(n \log n)$.

Para isso, $\text{DFT}(A) = \text{DFT}(a_0, \dots, a_{n-1}) = (y_0, \dots, y_{n-1}) = (A(w_n^0), \dots, A(w_n^{n-1}))$. Ademais, defina $\text{InverseDFT}(y_0, \dots, y_{n-1}) = (a_0, \dots, a_{n-1})$. Note que $AB = \text{InverseDFT}(\text{DFT}(AB)) = \text{InverseDFT}(\text{DFT}(A)\text{DFT}(B))$ (em que a multiplicação de dentro é component-wise). Sendo assim, a ideia é fazer a DFT e a InverseDFT em $O(n \log n)$, para daí obter AB .

Essa FFT não é nada boa para multiplicações envolvendo coeficientes altos. Vão ter muitos erros de imprecisão. Geralmente se não tiver gente perto de 10^{15} (pra double) ou 10^{18} (pra long double) tá de boa.

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1)
                j ^= bit;
            j ^= bit;
        }

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (
            invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+
                    j+len/2] * w;

```

```

                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const&
    a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()),
        fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    ;
    return result;
}

```

5.13.1 Number theoretic transform

Objetivo: multiplicar polinômios agora com os coeficientes módulo p .

Solução: para $p = 73400233$, $w_{2^{20}} = 5$ é uma 2^{20} -ésima raiz primitiva. Podemos, então, utilizar a seguinte FFT:

```

/**
 * NTT eh o mesmo que FFT, so que com
 * um *n-th root* diferente
 * MOD tem que ser um primo do tipo p =
 *   c * 2^k + 1
 * Com isso, a n-th root existe pra n =
 *   2^k.
 * Essa n-th root é g^c, onde g é um *
 * primitive root* de 2^k
 */

namespace NTT {
const int mod = 998244353;//
          998244353 7340033
const int root = 363395222;// 15311432
          5

```

```

const int root_1 = 704923114; //
469870224 4404020
const int root_pw = 1 << 19; // 1 <<
23; 1 << 20;
// ~5*10^5 ~8*10^6 ~10^6

int fast_pow(int a, int b, int m) {
    int ans = 1;
    while(b) {
        if(b&1) ans = 1LL * ans * a % m;
        a = 1LL * a * a % m;
        b >>= 1;
    }
    return ans;
}

int inverse(int x, int m) { return
fast_pow(x, m - 2, m); }

/* INICIO */
/** So quando as constantes acima nao
forem suficientes */

map<int, int> factor(int n) {
    map<int, int> ans;
    for(int i = 2; i * i <= n; ++i) {
        while(n%i == 0) {
            ans[i]++;
            n /= i;
        }
    }
    if(n > 1) ans[n]++;
    return ans;
}

int Phi(int n) {
    auto fact = factor(n);
    int ans = 1;

    for(auto [a, b] : fact) {
        ans *= fast_pow(a, b, n + 1) -
fast_pow(a, b-1, n + 1);
    }
    return ans;
}

int prim_root(int n) {
    int phi = Phi(n);
    auto fact = factor(phi);

    for (int res=2; res<=n; ++res) {
        if(__gcd(res, n) > 1) continue;
        bool ok = true;
        for (auto [a, b] : fact) {
            ok &= fast_pow(res, phi / a
, n) != 1;
            if(!ok) break;
        }
        if (ok) return res;
    }
    return -1;
}

```

```

}

// Generates NTT constants for any Mod
and prints it on the screen
void generate(int Mod) {

    int n = 1;
    int aux = Mod-1;
    while((aux&1) == 0) aux >>= 1, n
<<= 1;
    int c = aux;

    // g = primitive root de Mod
    int g = prim_root(Mod);
    if(g == -1) {
        printf("No constants could be
found, cant find primitive
root of %d\n", n);
        return;
    }
    int root = fast_pow(g, c, Mod);
    int root_1 = inverse(root, Mod);
    printf("mod = %d, root = %d, root_1
= %d root_pw = %d\n", Mod, root
, root_1, n);
    assert(fast_pow(root, n, Mod) == 1)
;
    // g^c
}

/* FIM */

inline void ntt(vector<int>& a, bool
invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++)
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<=
1) {
        int wlen = invert ? root_1 :
root;

        for (int i = len; i < root_pw;
i <<= 1)
            wlen = 1LL * wlen * wlen %
mod;

        for (int i = 0; i < n; i += len
) {
            int w = 1;
            for (int j = 0; j < len /

```



```

        2; j++) {
            int u = a[i+j], v = 1LL
                * a[i+j+len/2] * w
                % mod;
            a[i+j] = u + v < mod ?
                u + v : u + v - mod;
            a[i+j+len/2] = u - v >=
                0 ? u - v : u - v +
                mod;
            w = 1LL * w * wlen %
                mod;
        }
    }

    if (invert) {
        int n1 = inverse(n, mod);
        for (int & x : a)
            x = 1LL * x * n1 % mod;
    }
}

vector<int> multiply(vector<int> const&
    a, vector<int> const& b) {
    vector<int> ca(a.begin(), a.end()),
        cb(b.begin(), b.end());
    int n = 1;
    while(n < (int) max(a.size(), b.
        size())) n <<= 1;
    n <<= 1;

    ca.resize(n); cb.resize(n);

    ntt(ca, false); ntt(cb, false);

    for (int i = 0; i < n; i++) ca[i] =
        1LL * ca[i] * cb[i] % mod;
    ntt(ca, true);

    return ca;
}
};

```

Para módulos da forma $p = c2^k + 1$ (com p primo), g^c é uma 2^k -ésima raiz da unidade, em que g é uma raiz primitiva módulo p .

Atenção: Se em algum momento ela tentar calcular um fft de um vetor de tamanho maior que 2^k , vai bugar!

Como calcular NTT com módulo M arbitrário? Uma opção é usar teorema chinês dos restos em primos da forma $c2^k + 1$

Uma outra opção é decompor os polinômios $A(x)$ e $B(x)$ em $A = A_1 + A_2 \cdot C$ e $B = B_1 + B_2 \cdot C$ em que $C \approx \sqrt{M}$ (de modo que os coeficientes de cada um deles sejam pequenos) e daí

$$A \cdot B = A_1 \cdot B_1 + (A_1 \cdot B_2 + A_2 \cdot B_1) \cdot C + (A_2 \cdot B_2) \cdot C^2$$

ou seja, com isso, os coeficientes calculados (de cada produto $A_i B_j$) não passam de $n \cdot M$.

Problema 5.13.1: Número de pares de números com uma dada soma.

Solução: Eleve ao quadrado o polinômio de frequências.

Problema 5.13.2: Produto interno entre array a e shifts cíclicos de array b

Solução: Reverte a , completa com zeros, apenda b em b e calcula o produto entre os novos a e b . Na array c , $c[k] =$ produto escalar entre a e o left shift cíclico $k-(n-1)$ de b (com k indo de $n-1$ a $2n-2$).

```

// NAO E exatamente esse problema
// recebo duas fitas de DNA |s| >= |r|
// determinar r.size() - maior produto
// interno
// ou seja eu vou deslizando r em s (
// sem passar dos limites)
#include <bits/stdc++.h>

using namespace std;

using cd = complex<double>;
const double PI = acos(-1);
typedef long long int lli;

void fft(vector<cd>& a, bool invert) {}

vector<lli> multiply(vector<lli> const&
    a, vector<lli> const& b) {}

int conversion_map[200];

int main() {
    string s;
    string r;
    cin >> s;
    cin >> r;
    conversion_map['A'] = 0;
    conversion_map['C'] = 1;
    conversion_map['T'] = 2;
    conversion_map['G'] = 3;

    vector<lli> pols_s[4];
    vector<lli> pols_r[4];
    vector<lli> prods[4];
    for (int i = 0; i < s.size(); i++)
    {
        int atual = conversion_map[s[i]];
        for (int j = 0; j < 4; j++) {
            pols_s[j].push_back(atual
                == j);
        }
    }

    for (int i = 0; i < r.size(); i++)
    {
        int atual = conversion_map[r[i]];
        for (int j = 0; j < 4; j++) {
            pols_r[j].push_back(atual

```

```

        == j);
    }
}

for (int j = 0; j < 4; j++) {
    reverse(pols_r[j].begin(),
            pols_r[j].end());
    prods[j] = multiply(pols_s[j],
                        pols_r[j]);
    if (prods[j].size() < s.size()
        - 1) prods[j].resize(s.size()
                              - 1);
}

int max_ans = -1;

for (int i = r.size() - 1; i <= s.
    size() - 1; i++) {
    int curr_ans = 0;
    for (int j = 0; j < 4; j++) {
        curr_ans += prods[j][i];
    }
    if (curr_ans > max_ans) {
        max_ans = curr_ans;
    } else if (curr_ans < max_ans)
        continue;
}

cout << r.size() - max_ans << "\n";
}

```

Problema 5.13.3: Temos duas fitas de booleanos a e b . Queremos encontrar o número de maneiras de aplicar uma na outra de modo que nenhum par de uns estejam na mesma posição.

Solução: análoga a 5.13.2.

Problema 5.13.4: Dado um texto T , um padrão P (os 2 de letra minúscula), calcular todas as ocorrências do padrão no texto.

Solução: Criamos

$$A(x) = a_0 + \dots + a_{n-1}x^{n-1}, n = |T|$$

com $a_i = e^{i\alpha_i}$ onde $\alpha_i = \frac{2\pi T[i]}{26}$

$$B(x) = b_0 + \dots + b_{m-1}x^{m-1}, m = |P|$$

com $b_i = e^{-i\beta_i}$ onde $\beta_i = \frac{2\pi P[m-i-1]}{26}$

Agora, considere $C(x) = A(x) \cdot B(x)$. Se $c_{m-1-i} = m$, é porque teve um match a partir de $T[i]$.

Problema 5.13.5: Permitindo wildcards no padrão (letras que dão match com qualquer uma)

Solução: basta setar $b_i = 0$ sempre que $P[m-i-1] = *$. Aí se o padrão tiver x wildcards, então $c_{m-1-i} = m - x$ indica que a partir do índice $T[i]$ coincidem.

5.13.2 FFT 2D

```

const int B = 12; // nesse problema os
pols envolvidos tinham no max x^800
y^800

```

```

const int N = 1 << B;

cd roots[N];
int inv[N];

cd get_root(int k, int n) { return
    roots[k * (N / n)]; }

void precalc() {
    for (int i = 0; i < N; i++) {
        double ang = 2 * PI * i / N;
        roots[i] = cd(cos(ang), sin(ang));
    }
}

bool bit(int mask, int i) { return (
    mask >> i) & 1; }

void precalc_inv(int n) {
    int b = 0;
    while ((1 << b) < n)
        ++b;
    assert((1 << b) == n);
    assert(b <= B);

    inv[0] = 0;
    int hb = -1;
    for (int i = 1; i < n; ++i) {
        if (bit(i, hb + 1)) {
            ++hb;
        }
        inv[i] = inv[i ^ (1 << hb)] ^
            (1 << (b - hb - 1));
    }
}

// ****copia e cola fft aq

void fft_2d(vector<vector<cd>>& a, bool
    rev) {
    precalc_inv((a[0].size()));
    for (auto& row : a) {
        fft(row, rev);
    }
    precalc_inv((a.size()));
    for (int j = 0; j < a.front().size()
        (); j++) {
        vector<cd> col;
        for (int i = 0; i < a.size(); i
            ++){
            col.push_back(a[i][j]);
        }
        fft(col, rev);
        for (int i = 0; i < a.size(); i
            ++){
            a[i][j] = col[i];
        }
    }
}

vector<vector<cd>> mult(vector<vector<
    cd>> x, vector<vector<cd>> y) {

```

```

    int b_rows = 0;
    while ((1 << b_rows) <= max(x.size() , y.size()))
        ++b_rows;
    ++b_rows;

    int b_cols = 0;
    while ((1 << b_cols) <= max(x.front().size() , y.front().size()))
        ++b_cols;
    ++b_cols;

    x.resize(1 << b_rows);
    y.resize(1 << b_rows);
    for (auto& row : x) {
        row.resize(1 << b_cols , 0);
    }
    for (auto& row : y) {
        row.resize(1 << b_cols , 0);
    }

    fft_2d(x, 0);
    fft_2d(y, 0);

    for (int i = 0; i < x.size(); i++)
        for (int j = 0; j < x[i].size(); j++)
            x[i][j] *= y[i][j];

    fft_2d(x, 1);

    return x;
}

```

6 Combinatória

6.1 Fatoriais e binomiais

Problema 6.1.1: Como calcular $C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$?

Solução: Você pode calcular cada fatorial. Você pode calcular o produto a partir de um certo ponto no numerador e depois dividir pelo fatorial que sobrou. Você pode utilizar a recorrência $C(n, k) = C(n-1, k-1) + C(n-1, k)$. Você pode ter os fatoriais precomputados e calcular em $O(1)$.

Propriedades:

$$\begin{aligned}\binom{n}{k} &= \binom{n}{n-k} \\ \binom{n}{k} &= \frac{n}{k} \binom{n-1}{k-1} \\ \binom{n}{k} &= \binom{n-1}{k-1} + \binom{n-1}{k} \\ \sum_{k=0}^n \binom{n}{k} &= 2^n \\ \sum_{m=0}^n \binom{m}{k} &= \binom{n+1}{k+1} \\ \sum_{k=0}^m \binom{n+k}{k} &= \binom{n+m+1}{m} \\ \sum_{k=0}^n \binom{n}{k}^2 &= \binom{2n}{n} \\ \sum_{k=0}^n k \binom{n}{k} &= n \cdot 2^{n-1} \\ \sum_{k=0}^n \binom{n-k}{k} &= F_{n+1} \\ (a+b)^n &= \sum_{k=0}^n \binom{n}{k} a^k b^{n-k} \\ \binom{m+n}{r} &= \sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} \\ \sum_{k=0}^m (-1)^k \binom{m}{k} &= (-1)^m \binom{m-1}{m}\end{aligned}$$

Obs.: Se o "denominador" do coeficiente binomial for maior que o "numerador", definimo-lo como zero.

Problema 6.1.2: Calcular $C(n, k) \bmod m$.

Solução: fatorar m em potências de primo e juntar tudo com o teorema chinês dos restos. Para achar $C(n, k) \bmod p^e$, seja $c(x)$ o maior inteiro tal que $p^{c(x)} | x!$. Seja $g(x) = \frac{x!}{p^{c(x)}}$. Você pode calcular os g e os c com dp por exemplo. Aí no binomial você teria que lidar com

$$\frac{g(n)}{g(k)g(n-k)} p^{c(n)-c(k)-c(n-k)}$$

Como os g agora são coprimos com p dá pra calcular de boa o mod deles por potência de primo. E, quanto à outra parte, é só multiplicar no final.

Problema 6.1.3: $C(n, k) \bmod m$ pra n grande e m pequeno.

Solução 1 (m primo): Usa as técnicas do problema 2, que leva $O(m \log_m n)$.

Solução 2 (m primo): Usa o teorema de Lucas. Esse teorema diz que se $n = n_l p^l + \dots + n_0$ e $k = k_l p^l + \dots + k_0$ forem a expressão de n e k na base p , então

$$\binom{n}{k} \equiv \prod_{i=0}^l \binom{n_i}{k_i} \pmod{p}$$

Solução 3 (m livre de quadrados, fatores pequenos): Usa o teorema de Lucas e combina tudo com o teorema chinês dos restos.

Solução 4 (m qualquer): Usa o teorema de Lucas generalizado e junta com o teorema chinês dos restos. Seja $(k!)_p$ o produto de todos os inteiros menores ou iguais a k não divisíveis por p . Suponha que queremos achar $\binom{n}{m}$ (com $r = n - m \geq 0$) módulo p^f . Escreva $n = n_0 + n_1 p + \dots + n_s p^s$ em base p . Seja N_j o menor resíduo positivo de $\lfloor n/p^j \rfloor \pmod{p^f}$ (de modo que $N_j = n_j + \dots + n_{j+f-1} p^{f-1}$). Defina de maneira análoga m_j, M_j, r_j, R_j . Seja e_j o número de índices $i \geq j$ tais que $n_i < m_i$. Então:

$$\frac{1}{p^{e_0}} \binom{n}{m} \equiv (\pm 1)^{e_{f-1}} \prod_{i=0}^s \frac{(N_i!)_p}{(M_i!)_p (R_i!)_p} \pmod{p^f}$$

onde o ± 1 é -1 , exceto no caso $p = 2$ e $f \geq 3$.

Problema 6.1.4: Você quer ir de (a, b) para (c, d) indo apenas pra cima/baixo/esq/dir. Qual o número de caminhos com a menor distância?

Solução: Seja $\Delta_x = |a - c|$ e $\Delta_y = |b - d|$. Então, a resposta é $\binom{\Delta_x + \Delta_y}{\Delta_x}$.

Problema 6.1.5: Você tem uma array de $n = 10^5$ e vai processar $m = 10^5$ queries que adicionam $\binom{k}{k}$, $\binom{k+1}{k}, \dots$ no intervalo $[l, r]$. Sabendo que $k \leq 100$, dizer como a array fica no final.

Solução: Faça mais 101 arrays de "subtrações" de prefixo. Diga que $vet[0][i] = a[i] - a[i-1]$ e $vet[i][j] = vet[i-1][j] - vet[i-1][j-1]$. Se uma query envolve um k arbitrário, incremente $vet[k][l]$ e decrescente $vet[k][r+1]$. Depois de em todas as queries fazer isso, para cada k , reconstrua a array original a partir de $vet[k]$. Você vai obter várias arrays modificadas. Cada uma delas considera todas as queries para determinado k agindo no vetor. Então você soma todas elas e ajusta.

Problema 6.1.6: Você quer saber para quantos/quais k o número $\binom{n}{k}$ é ímpar.

Solução: Pelo teorema de Lucas, para ser ímpar, a cada bit 1 de k , não pode corresponder um bit 0 de n . Em outras palavras, cada bit 0 de n corresponderá

a um bit 0 de k . E pra cada bit 1 de n , o bit correspondente de k não importa. Daí, você pode iterar de 0 até n e pegar os k que satisfazem o que queremos com isso. E para pegar a quantidade é só 2^{n_1} onde n_1 é o número de bits 1 da representação decimal de n .

Problema 6.1.7: Dada uma string de parêntesis, como determinar o número de substrings (não necessariamente contíguas) iguais a $()$, $(())$, $((()))$, etc?

Solução: Para a string $(((...)))$ com x (e y), a resposta é $\binom{x+y}{x}$ (por ex via vandermonde). Agora, itere pelos (da string e para cada um deles, conte o número de substrings em que este é o último opening bracket. Para isso, suponha que tem x opening brackets (estritamente) antes dele e y closing brackets depois. Aí como esse (atual deve estar incluído na resposta, então queremos contar as substrings de $(((...))$). Então ponha uma lista no último opening bracket e vá somando as escolhas de $r - 1$ opening brackets e r closing brackets e some. Pela mesma lógica de vandermonde, agora a resposta é $\binom{x+y}{x+1}$.

Problema 6.1.8: Tem uma string s de $n \leq 10^6$ letras de a ate z. Cada conjunto de letras contíguas é uma "colônia" da especie de bacteria correspondente a essa letra (então número de espécies é menor ou igual a 26 e pode ter mais de uma colônia de uma dada espécie). A cada instante ocorre um ataque. Um ataque é tipo "aabb" vira "aaab" e é sempre um caractere. A pergunta é o número de possíveis estados que podem ocorrer. Por exemplo, se for "aab", temos "aab", "aaa", "abb", "bbb", então a resposta é 5;

Solução: Converta cada colonia para um caractere por ex "aaaabbcccc" vira "abc". É óbvio que uma string t de n caracteres é um estado válido se, e só se, ao eu converter cada colonia dela pra um caractere, isso é uma substring do "abc". Ou seja, dada uma substring de "abc", basta eu ver quantas strings t são tais que a compressão dessa string t seja essa substring. Digamos que essa substring tenha m caracteres. É fácil contar as strings t porque são o número de soluções de $x_1 + \dots + x_m = n$ com $x_i \geq 1$. Ou seja, $\binom{n-1}{m-1}$. Então temos que contar o número de substrings (não necessariamente contíguas) de "abc" (isto é, da compressão de s) que não possuem dois caracteres consecutivos iguais e que tenham " m " caracteres para cada valor de " m ". Dá pra fazer isso com uma $dp[c][l]$ - número de substrings de tamanho l que terminam em c . $dp[c][l] = 1 + soma[c != ci] dp[c, l-1]$

Problema 6.1.9: Tem n pontos com coordenadas inteiras no plano. Pra cada ponto, ou você põe uma reta horizontal passando por ele, ou uma vertical, ou você não faz nada. Dadas as coordenadas, qual o número de desenhos?

Solução: Imagina que você tem um grafo em que os pontos são vértices e tem uma aresta se e só se tiverem o mesmo x ou o mesmo y . A resposta é claramente o produto das respostas pra cada componente. Em uma dada componente, se nela não tiver nenhum "ciclo" (não ciclo, mas 4 pontos que formam um retângulo paralelo aos eixos coordenados), então a resposta pra ela é $2^{x+y} - 1$. Se tiver, é 2^{x+y} , em que x é a quan-

tidade de coordenadas x distintas e y analogamente.

Problema 6.1.10: Palitos e bolinhas. Você tem $x_i \geq 0$ e quer saber o número de soluções de $x_1 + \dots + x_n = N$.

Solução: $\binom{N+n-1}{n-1}$

Problema 6.1.11: Palitos e bolinhas estendido. Você tem $0 \leq x_i \leq r_i$ e quer saber o número de soluções de $x_1 + \dots + x_n = N$.

Solução: Argumenta por inclusão e exclusão no palitos e bolinhas normal. Vai chegar em

$$\sum_{S \subseteq \{1, 2, \dots, n\}} (-1)^{|S|} \binom{N+n-1 - \sum_{i \in S} r_i + 1}{n-1}$$

No caso particular em que $r_i = r$, temos

$$\sum_{k=0}^{\lfloor N/(r+1) \rfloor} (-1)^k \binom{n}{k} \binom{N-k(r+1)+n-1}{n-1}$$

Problema 6.1.12: Você tem um sanduíche de $N \leq 10^{18}$ metros e quer cortar no menor número de pedaços com comprimento positivo que é tal que é possível que cada pedaço fique com tamanho $\leq K \leq 10^{18} \bmod M \leq 10^6$

Solução: O número de pedaços q satisfaz $\lceil N/q \rceil \leq K$. Na verdade queremos o menor q que satisfaz isso. Esse q é $\lceil N/K \rceil$. Se o tamanho do sanduíche fosse qK , a resposta seria 1. Mas o tamanho dele é N . Note que $qK - N$, ou seja, estamos descendo do menor múltiplo de K maior ou igual a N para o próprio N . Então essa diminuição é menor que K . Ou seja, basta resolvermos $x_1 + \dots + x_q = qK - N$ em que $x_i < K$ (mas esta é uma restrição desnecessária, visto que $qK - N < K$), onde x_i representa o quanto tiramos do sanduíche i . Por isso, a resposta final é $\binom{qK-N+q-1}{q-1}$. Para calcular isso, temos binomiais muito grandes e arbitrários. Agora fatora o M e usa o teorema de lucas generalizado em cada potência de primo e junta tudo com o teorema chinês dos restos.

6.2 Números de catalão

Incluindo o zero,

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

de modo que os primeiros números sejam 1, 1, 2, 5, 14, 42, 132, 429, ...

O número C_n é a solução para:

1. Número de sequências de parêntesis balanceadas
2. Número de árvores binárias enraizadas completas com $n+1$ folhas. Uma árvore enraizada é cheia se cada vértice ou não tem filho ou tem dois filhos.
3. Número de maneiras de parentesear completamente $n+1$ fatores.
4. Número de triangulações de um polígono convexo de $n+2$ lados.
5. Número de maneiras de conectar $2n$ pontos num círculo para formar n cordas disjuntas.

6. Número de árvores binárias completas não isomórficas com n nós internos (aqueles que têm algum filho)
7. Número de caminhos de $(0,0)$ até (n,n) que não passam por cima da diagonal
8. Número de permutações de tamanho n que podem ser stack sorted, isto é. não tem $i < j < k$ satisfazendo $p_i < p_j < p_k$.
9. Número de partições non-crossing de um conjunto de n
10. Número de maneiras de cobrir a escala $1 \dots n$ usando n retângulos (a escada consiste de n colunas, onde a i -ésima possui altura i)

elementos

Rekursivamente, podemos definir $C_0 = C_1 = 1$ e para $n \geq 2$:

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$$

6.3 Princípio da Inclusão e Exclusão

Sejam n um inteiro positivo e $A_1, \dots, A_n$ conjuntos. Temos

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq J \subset \{1,2,\dots,n\}} (-1)^{|J|-1} \left| \bigcap_{j \in J} A_j \right|$$

Problema 6.3.1: Dado n e r , determine o número de inteiros de 1 a r relativamente primos com n .

Solução: Sejam $p_1, \dots, p_k$ os fatores primos de n . O número de caras divisível por x no intervalo é $\lfloor r/x \rfloor$. Então faz inclusão exclusão com os primos. Demoramos $O(\sqrt{n})$ para fatorar e iteremos pelos 2^k subconjuntos de p 's.

```
int solve (int n, int r) {
    vector<int> p;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            p.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        p.push_back (n);

    int sum = 0;
    for (int msk=1; msk<(1<<p.size());
        ++msk) {
        int mult = 1,
            bits = 0;
        for (int i=0; i<(int)p.size();
            ++i)
            if (msk & (1<<i)) {
```

```
                ++bits;
                mult *= p[i];
            }

        int cur = r / mult;
        if (bits % 2 == 1)
            sum += cur;
        else
            sum -= cur;
    }

    return r - sum;
}
```

Essa solução é $O(\sqrt{n})$.

Problema 6.3.2: Dados n números a_i e um número r , queremos os inteiros no intervalo $[1, r]$ que são múltiplos de pelo menos um a_i .

Solução: É quase idêntico ao problema anterior. Só que dada uma máscara precisamos pegar o lcm dos caras que compõem ela, o que leva $O(n \log r)$, de modo que a complexidade final seja $O(2^n n \log r)$

Problema 6.3.3: Número de strings que satisfazem um padrão: são dados n padrões de mesmo comprimento que consistem de letras de a a z ou interrogações. A tarefa é contar o número de strings que dão match em exatamente k dos padrões (primeiro problema) e em pelo menos k dos padrões (segundo problema).

Solução: É fácil verificar o número de strings que satisfazem um determinado conjunto de padrões. Mas agora, fixe X um subconjunto dos padrões que consiste de k padrões. Temos que contar o número de strings que satisfazem apenas X e não satisfazem nenhum outro padrão. Para isso, usamos o princípio da inclusão e exclusão: percorra todos os Y que contenham X e faça inclusão e exclusão da seguinte maneira:

$$\text{ans}(X) = \sum_{Y \supseteq X} (-1)^{|Y|-k} f(Y)$$

onde $f(Y)$ é o número de strings que dão match em Y . E para pegar a resposta final:

$$\text{ans} = \sum_{X: |X|=k} \text{ans}(X)$$

Essa solução tem assintótica de $O(3^k \cdot k)$. Para melhorá-la note que cada $\text{ans}(X)$ compartilha computações acerca dos conjuntos Y . Dá pra transformar a fórmula nisso:

$$\text{ans} = \sum_{Y: |Y| \geq k} (-1)^{|Y|-k} \cdot \binom{|Y|}{k} \cdot f(Y)$$

de modo que tenhamos uma assintótica $O(2^k \cdot k)$. Para a segunda versão do problema, note que a parte multiplicando $f(Y)$ seria

$$(-1)^{|Y|-k} \binom{|Y|}{k} + \dots + (-1)^{|Y|-|Y|} \binom{|Y|}{|Y|}$$

mas uma de nossas identidades combinatórias simplifica isso para

$$(-1)^{|Y|-k} \binom{|Y|-1}{|Y|-k}$$

então podemos calcular tudo em $O(2^k \cdot k)$

$$\text{ans} = \sum_{Y: |Y| \geq k} (-1)^{|Y|-k} \binom{|Y|-1}{|Y|-k} f(Y)$$

Problema 6.3.4: Tem um campo $n \times m$ e k de suas células são paredes intransponíveis. Um robô tá na célula $(1, 1)$ e pode ir apenas para direita ou esquerda. Eventualmente ele precisa ir para a célula (n, m) evitando todos os obstáculos. Qual o número de maneiras que ele pode fazer isso? Assuma $n, m \leq 10^9$ e $k \leq 100$

Solução: Coloque $(1, 1)$ no início e (n, m) no final da array de obstáculos. Seja $d[i]$ o número de maneiras de ir do ponto inicial até o obstáculo i da array sem pisar em nenhum outro obstáculo. Nossa resposta seria o último elemento de d . Primeiro, calcule normalmente o número de maneiras de ir da cela inicial até a cela i . Agora, desconte para cada $0 < t < i$ o número de maneiras de chegar até i passando por algum(ns) obstáculos, dentre os quais o primeiro foi o t . Esse número é $dp[t]$ multiplicado pelo número de caminhos para chegar até o i . A complexidade final seria $O(k^2)$.

Problema 6.3.5: Número de quádruplas coprimas. Sejam $a_1, \dots, a_n$ números dados. Calcule o número de maneiras de escolher quatro números de modo que seu gcd é 1.

Solução: Computaremos o número de quádruplas em que o gcd é maior que um. Itere por todos os grupos de quatro números divisíveis por d . Temos:

$$\text{ans} = \sum_{d \geq 2} (-1)^{\deg(d)-1} f(d)$$

onde $\deg(d)$ é o número de primos na fatoração de d e $f(d)$ é o número de quádruplas divisíveis por d . Para calcular $f(d)$ só precisa contar o número de múltiplos de d e escolher quatro deles.

Problema 6.3.6: Número de triplas harmônicas. Seja $n \leq 10^6$. Você precisa computar o número de triplas $2 \leq a < b < c \leq n$ que satisfazem uma das seguintes condições:

1. ou $\gcd(a, b) = \gcd(a, c) = \gcd(b, c) = 1$
2. ou $\gcd(a, b) > 1, \gcd(a, c) > 1, \gcd(b, c) > 1$

Primeiramente, considere o problema ao contrário i.e. contar o número de triplas não harmônicas. Uma tripla não harmônica é composta por um par de coprimos e um terceiro número que não é coprimo com pelo menos um cara do par. Então o número de triplas não harmônicas que contêm i é igual ao número de inteiros de 2 até n que são coprimos com i multiplicado pelo número de inteiros que não são coprimos com i . Mas temos que dividir por 2 por que fizemos contagens duplas para valores diferentes de i . Para calcular o número de coprimos a i no intervalo de 2 a n ,

uma solução mais rápida do que foi mencionada seria a seguinte:

1. Encontre todos os números em $[2, n]$ tais que sua fatoração não inclui nenhum fator primo duas vezes. Durante o crivo, podemos manter uma array $\text{deg}[i]$ que diz o número de primos na fatoração de i e uma array $\text{good}[i]$ que marca se cada i contém cada fator no máximo uma vez (1 ou 0). Ao iterar de 2 a n , se chegarmos em um número que tem $\text{deg} = 0$, então ele é primo e seu deg vira 1. Durante o crivo, vamos iterar i de 2 a n . Quando formos processar um primo, iteramos por todos os seus múltiplos e aumentamos o seu $\text{deg}[i]$. Se algum desses múltiplos for múltiplo do quadrado de i , colocamos seu good como falso.
2. Precisamos calcular a resposta para todo i de 2 até n - os números não coprimos com i . Para isso, iteramos por um produto de primos da sua fatoração e adicionamos ou subtraímos seus múltiplos. Então digamos que estejamos processando um número i com $\text{good}[i] = 1$. Itere por todos os números que são múltiplos de i e adicione ou subtraia $\lfloor N/i \rfloor$ da $\text{cnt}[]$ de acordo com a paridade de $\text{deg}[i]$.

```
int n;
bool good[MAXN];
int deg[MAXN], cnt[MAXN];

long long solve() {
    memset(good, 1, sizeof good);
    memset(deg, 0, sizeof deg);
    memset(cnt, 0, sizeof cnt);

    long long ans_bad = 0;
    for (int i=2; i<=n; ++i) {
        if (good[i]) {
            if (deg[i] == 0) deg[i] = 1;
            for (int j=1; i*j<=n; ++j) {
                if (j > 1 && deg[i] == 1)
                    if (j % i == 0)
                        good[i*j] = false;
                    else
                        ++deg[i*j];
                cnt[i*j] += (n / i) * (deg[i]%2==1 ? +1 : -1);
            }
            ans_bad += (cnt[i] - 1) * 111 * (n-1 - cnt[i]);
        }
    }

    return (n-1) * 111 * (n-2) * (n-3) / 6 - ans_bad / 2;
}
```


A assintótica dessa solução é $O(n \log n)$.

Problema 6.3.7: Número de permutações caóticas de n elementos (aquelas que não contêm pontos fixos):

Solução: É possível calcular arredondando $n!/e$ para o inteiro mais próximo ou então calculando de maneira exata com a fórmula:

$$n! - \binom{n}{1}(n-1)! + \binom{n}{2}(n-2)! + \cdots \pm \binom{n}{n}(n-n)!$$

Para chegar nisso, dava pra ter usado inclusão e exclusão nos conjuntos A_k de permutações de tamanho n com um ponto fixo na posição k .

6.4 Teoria dos grupos

Seja G um grupo finito que age no conjunto X .

Dado $x \in X$, a órbita de x , denotada por $G \cdot x$, é o subconjunto de X que pode ser alcançado por ações de G em x i.e. $G \cdot x = \{g \cdot x : g \in G\}$. Note também que $x \in G \cdot y \iff G \cdot x = G \cdot y$. Com isso, é possível dividir X em suas órbitas. Ou seja, podemos dizer que X/G é o conjunto de todas as órbitas de X sobre a ação de G .

Defina também o estabilizador de $x \in X$ como o conjunto dos $g \in G$ tais que x é um de seus pontos fixos i.e. $G_x = \{g \in G : g \cdot x = x\}$. Em particular, note que $\forall x \in X$, G_x é um subgrupo de G . É fácil ver que este é um subgrupo normal, por isso há sentido em definir G/G_x .

Teorema da órbita e do estabilizador:

$$\forall x \in X, G \cdot x \cong G/G_x$$

Corolário: No caso finito, pelo teorema de Lagrange, temos $|G \cdot x| \cdot |G_x| = |G|$

Lema de Burnside: Defina $X^g = \{x \in X : g \cdot x = x\}$. Então

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

Se imaginarmos G como um grupo de permutações π que agem em X , podemos trocar o X^g por $I(\pi)$, o que representaria os elementos de x invariantes por determinada permutação.

Seja $C(\pi)$ o número de ciclos na decomposição em ciclos da permutação π . Seja Y um conjunto de cores. Seja Y^X o conjunto de funções $X \rightarrow Y$, isto é, que associam a cada elemento de X uma cor de Y .

Teorema da enumeração de Pólya:

$$|Y^X/G| = \frac{1}{|G|} \sum_{\pi \in G} |Y|^{C(\pi)}$$

Note que esse teorema é uma consequência do lema de Burnside, pois $|I(\pi)| = k^{C(\pi)}$.

Problema 6.4.1: Você tem k cores para colorir n bolinhas que vão formar um colar. Um colar é idêntico a outro se pode ser obtido por rotação. Qual a quantidade desses colares?

Solução: Seja $G \cong C_n$. Temos que G está agindo sobre colorações do conjunto X de bolas com um conjunto Y de cores i.e. G está agindo sobre Y^X . O

número de colorações é $|Y^X/G|$, que é o que queremos calcular. $|G| = n$. $|Y| = k$. Seja $\pi \in G$ uma rotação simples do colar, de modo que queremos determinar $C(\pi^k)$. Seja $g = \gcd(n, k)$. Note que π^k é composta por ciclos de tamanho n/g e que existem g desses ciclos. Então nossa resposta é:

$$\frac{1}{n} \sum_{i=0}^{n-1} k^{\gcd(i, n)}$$

Dá pra reescrever isso considerando que agora passamos por todos os valores de $\gcd(i, n)$. Para cada divisor d de n , o número de i tais que $d = \gcd(i, n)$ é claramente $\phi(n/d)$. Por isso, outra forma de escrever seria

$$\frac{1}{n} \sum_{d|n} \phi(n/d) k^d$$

Problema 6.4.2: Sejam n, m inteiros positivos com $n < m$. Considere um retângulo $n \times m$ onde cada casinha é preta ou branca. Agora pegue o lado longo desse retângulo e junte com o outro formando o cilindro. Agora dobre o cilindro de maneira que os círculos de baixo/cima se conectem e tenhamos um toro. Qual o número de toros distintos que podemos obter? O toro pode ser girado em torno dos círculinhos dele e também girado de fato.

Solução: Geralmente é difícil de obter um número explícito de classes de equivalência. Então a gente tem que fazer na marra. Pra isso, encontramos várias permutações básicas que geram o grupo G . É fácil ver que G é gerado por π_1 , π_2 e π_3 , onde π_1 shifta ciclicamente todas as linhas, π_2 shifta ciclicamente todas as colunas e π_3 gira a folha 180 graus. Então lembrando que nesse caso temos $|Y| = 2$, podemos obter o número de classes de equivalência (resposta do problema) utilizando o teorema de Pólya:

```
using Permutation = vector<int>;

void operator*=(Permutation& p,
Permutation const& q) {
    Permutation copy = p;
    for (int i = 0; i < p.size(); i++)
        p[i] = copy[q[i]];
}

int count_cycles(Permutation p) {
    int cnt = 0;
    for (int i = 0; i < p.size(); i++)
    {
        if (p[i] != -1) {
            cnt++;
            for (int j = i; p[j] !=
-1; j = p[j])
                p[j] = -1;
        }
    }
}

return cnt;
}
```

```

int solve(int n, int m) {
    Permutation p(n*m), p1(n*m), p2(n*m), p3(n*m);
    for (int i = 0; i < n*m; i++) {
        p[i] = i;
        p1[i] = (i % n + 1) % n + i / n * n;
        p2[i] = (i / n + 1) % m * n + i % n;
        p3[i] = (m - 1 - i / n) * n + (n - 1 - i % n);
    }

    set<Permutation> s;
    for (int i1 = 0; i1 < n; i1++) {
        for (int i2 = 0; i2 < m; i2++) {
            for (int i3 = 0; i3 < 2; i3++) {
                s.insert(p);
                p *= p3;
            }
            p *= p2;
        }
        p *= p1;
    }

    int sum = 0;
    for (Permutation const& p : s) {
        sum += 1 << count_cycles(p);
    }
    return sum / s.size();
}

```

6.5 K-Combinações

Na lib padrão do c++, tem a função next_permutation:

```

#include <iostream>
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> vec = {1, 2, 3};

    // Ordena o vetor em ordem crescente
    std::sort(vec.begin(), vec.end());

    do {
        // Imprime a permutacao atual
        for (int i : vec) {
            std::cout << i << " ";
        }
        std::cout << std::endl;
    } while (std::next_permutation(vec.begin(), vec.end()));

    return 0;
}

```

```

}

```

Dados inteiros positivos N e K , determinar os subconjuntos de $\{1, \dots, N\}$ de tamanho exatamente K .

Problema 6.5.1: Determine a próxima K -combinação (considerando ordem lexicográfica) considerando a primeira como sendo $1, 2, \dots, K$

Solução: Em $O(K)$

```

bool next_combination(vector<int>& a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false;
}

```

Problema 6.5.2: Gere todas as K -combinações de modo que combinações adjacentes difiram por no máximo um elemento

Solução 1: Em $O(2^n)$. Usa o fato de que se você gerar os 2^n graycodes de N bits e pegar apenas os de K bits ativos, então a diferença entre caras consecutivos vai ser que de um pro outro um bit apaga e um bit acende.

```

int gray_code(int n) {
    return n ^ (n >> 1);
}

int count_bits(int n) {
    int res = 0;
    for (; n; n >>= 1)
        res += n & 1;
    return res;
}

void all_combinations(int n, int k) {
    for (int i = 0; i < (1 << n); i++) {
        int cur = gray_code(i);
        if (count_bits(cur) == k) {
            for (int j = 0; j < n; j++) {
                if (cur & (1 << j))
                    cout << j + 1;
            }
            cout << "\n";
        }
    }
}

```

Solução 2: Em $O\left(N \cdot \binom{N}{K}\right)$, mas com constante alta

```

vector<int> ans;

```

```

void gen(int n, int k, int idx, bool
    rev) {
    if (k > n || k < 0)
        return;

    if (!n) {
        for (int i = 0; i < idx; ++i) {
            if (ans[i])
                cout << i + 1;
        }
        cout << "\n";
        return;
    }

    ans[idx] = rev;
    gen(n - 1, k - rev, idx + 1, false);
    ans[idx] = !rev;
    gen(n - 1, k - !rev, idx + 1, true);
}

void all_combinations(int n, int k) {
    ans.resize(n);
    gen(n, k, 0, false);
}

```

6.6 Extras

Problema 6.6.1: Encontre o número de maneiras de colocar K bispos num tabuleiro $n \times n$ de modo que não se ataquem

Solução: Numere as diagonais de maneira que as pretas tenham índices ímpares, as brancas tenham índices pares e as diagonais são numeradas em ordem do número de quadrados nela. Aqui um exemplo para um tabuleiro 5×5

1	2	5	6	9
2	5	6	9	8
5	6	9	8	7
6	9	8	7	4
9	8	7	4	3

Seja $d[i][j]$ o número de maneiras de colocar j bispos em diagonais com índices até o i e que têm a mesma cor da diagonal i . Dessa maneira, i varia de 0 a $2n - 1$ e j varia de 0 a k .

Podemos calcular $d[i][j]$ usando apenas os valores de $d[i - 2]$. Tem duas maneiras de calcular. Ou coloca todos os j bispos em diagonais anteriores, de modo que tenham $d[i - 2][j]$ maneiras de que isso aconteça. Ou então colocamos um bispo na diagonal i e $j - 1$ bispos em diagonais anteriores. O número de maneiras de fazer isso é igual ao número de quadrados na diagonal i menos $j - 1$, já que $j - 1$ bispos colocados em diagonais anteriores bloquearão um quadrado na diagonal atual. O número de quadrados na diagonal i é fácil de calcular:

```

int squares (int i) {
    if (i & 1)
        return i / 4 * 2 + 1;
    else
        return (i - 1) / 4 * 2 + 2;
}

```

O caso base é simples: $d[i][0] = d[1][1] = 1$. Tendo calculado os $D[i][j]$ a resposta é a soma dos $D[2n - 1][i] * D[2n - 2][k - i]$.

```

int bishop_placements(int N, int K)
{
    if (K > 2 * N - 1)
        return 0;

    vector<vector<int>> D(N * 2, vector<int>(K + 1));
    for (int i = 0; i < N * 2; ++i)
        D[i][0] = 1;
    D[1][1] = 1;
    for (int i = 2; i < N * 2; ++i)
        for (int j = 1; j <= K; ++j)
            D[i][j] = D[i - 2][j] + D[i - 2][j - 1] * (squares(i) - j + 1);

    int ans = 0;
    for (int i = 0; i <= K; ++i)
        ans += D[N * 2 - 1][i] * D[N * 2 - 2][K - i];
    return ans;
}

```

Problema 6.6.2: Encontrar a próxima sequência de parêntesis balanceada.

```

// inicialmente começa com (((...)))
// vai para a proxima em O(n)
// lembrando que sao C_n ao todo
bool next_balanced_sequence(string & s)
{
    int n = s.size();
    int depth = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (s[i] == '(')
            depth--;
        else
            depth++;

        if (s[i] == '(' && depth > 0) {
            depth--;
            int open = (n - i - 1 - depth) / 2;
            int close = n - i - 1 - open;
            string next = s.substr(0, i) + ')' + string(open, '(') + string(close, ')');
            s.swap(next);
            return true;
        }
    }
}

```

```

    }
    return false;
}

```

Problema 6.6.3: Encontrar índice dessa sequência na lista de sequências

Solução: Seja $d[i][j]$ onde i é o tamanho da sequência (semi-balanceada, isto é, tem parêntese que falta fechar) e j é o balanço atual (diferença entre opening e closing brackets) o número de sequências que correspondem a esse i e esse j .

Inicialmente, $d[0][0] = 1$, $d[0][j] = 0$, $j > 0$ e depois

$$d[i][j] = d[i-1][j-1] + d[i-1][j+1]$$

Então em $O(n^2)$ fazemos essa dp. Para calcular o índice de uma dada sequência, fazemos o seguinte. Primeiro fazemos o caso em que só tem um tipo de parêntese. Temos um contador $depth$ de profundidade que diz o quão nested nós estamos e iteramos pelos caracteres da sequência. Se $s[i] == ($, então incrementamos $depth$. Se o caractere atual for $)$, então incrementamos $d[2n-i-1][depth+1]$ na resposta e decrementamos $depth$.

Agora, se tiverem k tipos diferentes de brackets, então quando observamos o caractere atual $s[i]$, temos que iterar por todos os tipos de brackets que são menores que o caractere atual e tentar colocar esse bracket na sua posição atual (obtendo um novo balance $ndepth = depth \pm 1$) e daí adicionamos o número de maneiras de terminar a sequência (tamanho $2n-i-1$, balanço $ndepth$) à resposta:

$$d[2n-i-1][ndepth] \cdot k^{\frac{2n-i-1-ndepth}{2}}$$

Problema 6.6.4: Encontrando a k -ésima sequência.

Solução: Seja n o número de pares de brackets na sequência. No problema anterior computamos $d[i][j]$. Primeiro, com apenas um tipo de bracket: nós iteramos pelos caracteres na string que queremos gerar. Aqui também tem um contador $depth$. Em cada posição queremos decidir se usamos um opening ou closing bracket. Para ser um opening tem que ter $d[2n-i-1][depth+1] \geq k$. Incrementamos o contador $depth$ e vamos para o próximo. Caso contrário decrementamos k por $d[2n-i-1][depth+1]$, colocamos um closing bracket e continuamos

```

string kth_balanced(int n, int k) {
    vector<vector<int>>> d(2*n+1, vector<int>(n+1, 0));
    d[0][0] = 1;
    for (int i = 1; i <= 2*n; i++) {
        d[i][0] = d[i-1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i-1][j-1] + d[i-1][j+1];
        d[i][n] = d[i-1][n-1];
    }

    string ans;

```

```

    int depth = 0;
    for (int i = 0; i < 2*n; i++) {
        if (depth + 1 <= n && d[2*n-i-1][depth+1] >= k) {
            ans += '(';
            depth++;
        } else {
            ans += ')';
            if (depth + 1 <= n)
                k -= d[2*n-i-1][depth+1];
            depth--;
        }
    }
    return ans;
}

```

Agora, se tiverem k tipos de brackets, multiplicamos o valor de $d[2n-i-1][ndepth]$ por $k^{\frac{2n-i-1-ndepth}{2}}$ e levamos em conta o fato de que podem existir tipos de brackets diferentes para o próximo caractere.

Aqui uma implementação para dois tipos de bracket

```

string kth_balanced2(int n, int k) {
    vector<vector<int>>> d(2*n+1, vector<int>(n+1, 0));
    d[0][0] = 1;
    for (int i = 1; i <= 2*n; i++) {
        d[i][0] = d[i-1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i-1][j-1] + d[i-1][j+1];
        d[i][n] = d[i-1][n-1];
    }

    string ans;
    int shift, depth = 0;

    stack<char> st;
    for (int i = 0; i < 2*n; i++) {

        // '('
        shift = ((2*n-i-1-depth-1) / 2);
        ;
        if (shift >= 0 && depth + 1 <= n) {
            int cnt = d[2*n-i-1][depth+1] << shift;
            if (cnt >= k) {
                ans += '(';
                st.push('(');
                depth++;
                continue;
            }
            k -= cnt;
        }

        // ')'
        shift = ((2*n-i-1-depth+1) / 2);
        ;
    }
}

```

```

    if (shift >= 0 && depth && st.
        top() == '(') {
        int cnt = d[2*n-i-1][depth
            -1] << shift;
        if (cnt >= k) {
            ans += '(';
            st.pop();
            depth--;
            continue;
        }
        k -= cnt;
    }

    // '['
    shift = ((2*n-i-1-depth-1) / 2)
        ;
    if (shift >= 0 && depth + 1 <=
        n) {
        int cnt = d[2*n-i-1][depth
            +1] << shift;
        if (cnt >= k) {
            ans += '[';
            st.push('[');
            depth++;
            continue;
        }
        k -= cnt;
    }

    // ']'
    ans += ']';
    st.pop();
    depth--;
}
return ans;
}

```

Problema 6.6.5: Contando o número de grafos rotulados (isto é, cada vértice tem um rótulo de 1 a n) de n vértices. (não consideramos multi-edges nem directed edges).

Solução: Para cada conexão, decidimos se fazemos ela ou não: $G_n = 2^{n(n-1)/2}$;

Problema 6.6.6: Grafos rotulados conexos.

Solução: Digamos que a resposta é C_n . Primeiros focamos nos desconexos. Na verdade, primeiro focamos nos desconexos e enraizados. Um grafo enraizado é um em que eu enfatizo um vértice como sendo uma raiz (temos n possibilidades de escolher a raiz então é só dividir o número de grafos enraizados por n). O vértice raiz irá aparecer na componente conexa de tamanho $1, 2, \dots, n-1$. Existem $k \binom{n}{k} C_k G_{n-k}$ grafos tais que o vértice raiz está numa componente conexa com k vértices (existem n escolhe k maneiras de escolher k vértices para a componente, as quais podem ser conectadas de C_k maneiras, o vértice raiz pode ser qualquer um dos k vértices e os outros $n-k$ vértices podem ser conectados/desconectados de qualquer maneira). Então

$$C_n = G_n - \frac{1}{n} \sum_{k=1}^{n-1} k \binom{n}{k} C_k G_{n-k}$$

Problema 6.6.7: Grafos rotulados com k componentes conexas.

Solução: Seja $D[i][j]$ o número de grafos rotulados com i vértices e j componentes (para cada $i \leq n$ e cada $j \leq k$). Tomamos o último vértice (índice n). Esse vértice pertence a alguma componente. Se o tamanho dessa componente é s , então existem $\binom{n-1}{s-1}$ maneiras de escolher esse conjunto de vértices e C_s maneiras de conectá-los. Depois de remover essa componente do grafo temos $n-s$ vértices restando com $k-1$ componentes conexas. Daí,

$$D[n][k] = \sum_{s=1}^n \binom{n-1}{s-1} C_s D[n-s][k-1]$$

Problema 6.6.8: Números de Delanoy. Qual o número de caminhos de $(0,0)$ para (m,n) , indo pra cima, pra direita ou na diagonal? Como resolver a recorrência $D(n,m) = 1$ se $n = 0$ ou $m = 0$ e $D(n,m) = D(n-1,m) + D(m-1,n-1) + D(n,m-1)$ caso contrário?

Solução:

$$D(n,m) = \sum_{k=0}^{\min(m,n)} \binom{m}{k} \binom{n}{k} 2^k$$

Problema 6.6.9: Números de Schröder. Qual o número de caminhos S_n na mesma situação do problema acima só que agora em nenhum momento passam por cima da diagonal principal? Quantos dominós (retângulos 1×2 e 2×1) podemos arranjar em um formado de diamante asteca?

Solução: Com $S_0 = 1$ e $S_1 = 2$, satisfazem as recorrências

$$S_n = 3S_{n-1} + \sum_{k=1}^{n-2} S_k S_{n-k-1}$$

e

$$S_n = \frac{6n-3}{n+1} S_{n-1} - \frac{n-2}{n+1} S_{n-2}$$

O número de dominós é $2^{n(n+1)/2}$. A conexão com os números de Schröder é que um determinante com esses números dá isso, mas isso é irrelevante pra maratona provavelmente.

7 Teoria dos Jogos

7.1 Jogos em grafos arbitrários

Suponha que dois jogadores jogam um jogo num grafo arbitrário G . Os jogadores se movem em turnos e vão para um vértice adjacente. Dependendo do jogo, a pessoa que não puder se mover vai perder ou ganhar.

Consideramos um grafo dirigido arbitrário. A tarefa é determinado, dado um estado inicial, quem vai ganhar o jogo se os dois tiverem estratégias ótimas (ou então determinar que haverá um empate). Será encontrada uma solução para todos os vértices iniciais do grafo em $O(m)$ onde $m = \text{num de arestas}$.

Um vértice é ganhador se um player começando nele ganha o jogo se jogar de maneira ótima. Analogamente denotaremos o outro por perdedor. Para todos os vértices sem outgoing edges já sabemos sua condição de vitória/derrota. Além disso:

1. Se um vértice tem uma aresta que vai para um perdedor, então ele é ganhador
2. Se todas as arestas que saem de um vértice são vencedoras, então o vértice é perdedor
3. Se em algum ponto ainda existirem vértices indefinidos, então é um vértice de empate

Então dá pra fazer um algoritmo em $O(nm)$: roda pra todos os vértices aplicando essas regras. Mas dá pra reduzir pra $O(m)$.

Iteramos por todos os vértices que sabemos o estado inicial. Para cada um deles fazemos uma DFS que se move considerando as arestas ao contrário. Ela não vai entrar em vértices que já tem vitória/derrota definida. Se a pesquisa for de um vértice perdedor para um indefinido, então marca o indefinido como ganhador e continua a DFS dele. Se nós formos de um ganhador para um indefinido, então chegamos se todas as arestas desse indefinido vão para ganhadores. Dá pra fazer isso em $O(1)$ guardando o número de arestas que vão para vencedores para cada vértice.

```
vector<vector<int>>> adj_rev;

vector<bool> winning;
vector<bool> losing;
vector<bool> visited;
vector<int> degree;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj_rev[v]) {
        if (!visited[u]) {
            if (losing[v])
                winning[u] = true;
            else if (--degree[u] == 0)
                losing[u] = true;
            else
                continue;
            dfs(u);
        }
    }
}
```

```
}
}
```

Problema 7.1.1: Polícia e ladrão. Tem um tabuleiro $m \times n$. Algumas das células não podem ser entradas. As coordenadas iniciais do policial e do ladrão são conhecidas. Uma das células é a saída. Se o policial e o ladrão estão na mesma cela em qualquer momento, o policial ganha. Se o ladrão estiver na saída (sem que o policial esteja lá também), ele ganha. O policial pode andar nas 8 direções e o ladrão em 4. O policial e o ladrão alternam os movimentos, mas também podem skipar um movimento se quiserem.

Solução: construímos um grafo. O estado do jogo é definido pelas coordenadas do policial, do ladrão e de quem é a vez. Ou seja, um vértice do grafo é (P, T, P_{turn}) . Precisamos determinar quais vértices são perdedores e ganhadores. Se é a vez do policial, então o vértice é um winning se as coordenadas dos dois coincidirem e é losing se não for winning E o policial está na exit. Se é a vez do ladrão, então é losing se as coordenadas coincidirem e winning se não for losing e o ladrão estiver no exit vertex.

Sobre a implementação, é importante decidir se você vai construir o grafo explicitamente ou on the fly. O pro é que tem menos chance de ter erro e vai ser bem mais fácil de codar. O contra é que vai ser muito lento do que usando on the fly.

Aqui uma approach construindo explicitamente:

```
struct State {
    int P, T;
    bool Pstep;
};

vector<State> adj_rev[100][100][2]; //
[P][T][Pstep]
bool winning[100][100][2];
bool losing[100][100][2];
bool visited[100][100][2];
int degree[100][100][2];

void dfs(State v) {
    visited[v.P][v.T][v.Pstep] = true;
    for (State u : adj_rev[v.P][v.T][v.Pstep]) {
        if (!visited[u.P][u.T][u.Pstep]) {
            if (losing[v.P][v.T][v.Pstep])
                winning[u.P][u.T][u.Pstep] = true;
            else if (--degree[u.P][u.T][u.Pstep] == 0)
                losing[u.P][u.T][u.Pstep] = true;
            else
                continue;
            dfs(u);
        }
    }
}
```



```

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int P = 0; P < n*m; P++) {
        for (int T = 0; T < n*m; T++) {
            for (int Pstep = 0; Pstep
                <= 1; Pstep++) {
                int Px = P/m, Py = P%m,
                    Tx = T/m, Ty = T%m;
                if (a[Px][Py] == '*' || a[
                    Tx][Ty] == '*')
                    continue;

                bool& win = winning[P][
                    T][Pstep];
                bool& lose = losing[P][
                    T][Pstep];
                if (Pstep) {
                    win = Px==Tx && Py
                        ==Ty;
                    lose = !win && a[Tx
                        ][Ty] == 'E';
                } else {
                    lose = Px==Tx && Py
                        ==Ty;
                    win = !lose && a[Tx
                        ][Ty] == 'E';
                }
                if (win || lose)
                    continue;

                State st = {P, T, !Pstep
                    };
                adj_rev[P][T][Pstep].
                    push_back(st);
                st.Pstep = Pstep;
                degree[P][T][Pstep]++;

                const int dx[] = {-1,
                    0, 1, 0, -1, -1, 1,
                    1};
                const int dy[] = {0, 1,
                    0, -1, -1, 1, -1,
                    1};
                for (int d = 0; d < (
                    Pstep ? 8 : 4); d++)
                {
                    int PPx = Px, PPy =
                        Py, TTx = Tx,
                        TTy = Ty;
                    if (Pstep) {
                        PPx += dx[d];
                        PPy += dy[d];
                    } else {
                        TTx += dx[d];
                        TTy += dy[d];

```

```

                }

                if (PPx >= 0 && PPx
                    < n && PPy >= 0
                    && PPy < m && a
                        [PPx][PPy] != '*'
                        ' &&
                        TTx >= 0 && TTx
                            < n && TTy
                                >= 0 && TTy
                                    < m && a[TTx
                                        ][TTy] != '*'
                                        ')
                {
                    adj_rev[PPx*m+
                        PPy][TTx*m+
                        TTy][!Pstep
                            ].push_back(
                                st);
                    ++degree[P][T][
                        Pstep];
                }
            }
        }
    }

    for (int P = 0; P < n*m; P++) {
        for (int T = 0; T < n*m; T++) {
            for (int Pstep = 0; Pstep
                <= 1; Pstep++) {
                if ((winning[P][T][
                    Pstep] || losing[P][
                    T][Pstep]) && !
                    visited[P][T][Pstep
                        ])
                    dfs({P, T, (bool)
                        Pstep});
            }
        }
    }

    int P_st, T_st;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (a[i][j] == 'P')
                P_st = i*m+j;
            else if (a[i][j] == 'T')
                T_st = i*m+j;
        }
    }

    if (winning[P_st][T_st][true]) {
        cout << "Police catches the
            thief" << endl;
    } else if (losing[P_st][T_st][true
        ]) {
        cout << "The thief escapes" <<
            endl;
    } else {
        cout << "Draw" << endl;
    }
}

```


}
}

7.2 Teorema de Sprague-Grundy e Nim

O teorema descreve o jogo de dois players inicial i.e. aqueles em que os movimentos disponíveis e winning/losing depende apenas do estado do jogo. Ou seja, a única diferença entre os jogadores é quem se move primeiro.

Também assumimos que o jogo tem informação perfeita i.e. os jogadores sabem perfeitamente as regras e movimentos possíveis. Também assumimos que é finito i.e. depois de uma certa quantidade de movimentos um dos jogadores acabará numa losing position da qual não pode se mover. Então não há empate nesse jogo.

Esses jogos podem ser descritos completamente por um DAG.

7.2.1 Nim

Esse é um jogo que obedece tudo o que foi dito acima. Além disso, qualquer jogo imparcial two-layer de informação perfeita pode ser reduzido para o jogo de nim. Então estudar o nim resolve todos os jogos.

Você tem várias pilhas com várias pedras. Num movimento, um jogador deve escolher uma pilha e retirar uma quantidade inteira positiva de pedras dessa pilha. Ele perde se não tem mais pedras.

Teorema (Nim): O jogador atual tem uma estratégia vencedora se e só se a xor-sum das pilhas é não zero i.e. $s = a_1 \oplus \dots \oplus a_n \neq 0$. Além disso, o movimento vencedor de um dado estado consiste em reduzir alguma pilha de tamanho x para o tamanho $s \oplus x$.

Corolário: todo estado do nim pode ser substituído por outro com o mesmo xor. Quando estamos analisando um nim com várias pilhas podemos observar isso como substituindo por uma única pilha de tamanho s .

7.2.2 Teorema de Sprague-Grundy

Agora suponha que podemos adicionar pedras a uma pilha escolhida. As regras exatas de como e quando pode ocorrer esse aumento não nos interessam, mas o que importa é que essas regras devem deixar nosso jogo acíclico.

Lema: A adição de acréscimos a pedras não muda a maneira como estados de vitória/derrota são determinados. Ou seja, podemos ignorá-los.

Teorema: Considere um estado v de um jogo imparcial e sejam v_i os estados alcançáveis dele (com $i \in \{1, \dots, k\}$, $k \geq 0$). A esse estado podemos designar um jogo de nim completamente equivalente com uma pilha de tamanho x . O número x é dito o valor de Grundy ou nim-value do estado v . Esse número pode ser achado de maneira recursiva:

$$x = \text{mex}\{x_1, \dots, x_k\}$$

em que x_i são os valores de Grundy para os estados v_i e o mex pega o menor valor não negativo que não está no conjunto.

Aplicação: Para calcular o valor de Grundy de um estado, você deve

1. Pegar todas as transições a partir desse estado
2. Cada transição pode levar à soma de jogos independentes. Calcule o valor de Grundy para cada jogo independente e xor-sum neles.
3. Depois de calcular o valor de Grundy para cada transição, pega o mex dos números
4. Se o valor for zero, então o estado atual é losing. Se não for, é winning.

Dica: em tarefas específicas estudar a tabela de valores em busca de padrões é uma ótima ideia. Em muitos jogos onde uma análise teórica pode parecer difícil, os valores de Grundy são periódicos ou então tem uma forma compreensível. Em 99,9% dos casos dá pra provar o padrão por indução. Entretanto, ainda está aberto o problema de se existem regularidades em alguns casos.

Problema 7.2.1: Considere uma fita de tamanho $1 \times n$. Em um movimento, o jogador deve colocar uma cruz. Mas é proibido colocar duas cruces uma ao lado da outra. O jogador sem movimentos válidos perde.

Solução: Seja g o valor de Grundy. É trivial ver que

$$g(n) = \text{mex}(\{g(n-2)\} \cup \{g(i-2) \oplus g(n-i-1) | 2 \leq i \leq n-1\})$$

Com isso, temos uma solução $O(n^2)$. Na verdade, $g(n)$ tem período de tamanho 34 começando com $n = 52$.

8 Álgebra Linear

8.1 Equações lineares

Podemos resolver o sistema $Ax = b$ com o seguinte código (onde o vector de vector é a matriz a só que com a última coluna sendo b, o vector de resposta vai ser alterado e será retornado 0, 1 ou infinito correspondendo ao número de soluções do sistema):

```
d847fe
const double EPS = 1e-9;
const int INF = 2; // it doesn't
    actually have to be infinity or a
    big number

int gauss(vector<vector<double>>> a,
vector<double>& ans) {
    int n = (int)a.size();
    int m = (int)a[0].size() - 1;

    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m
        && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i)
            if (abs(a[i][col]) > abs(a[
                sel][col])) sel = i;
        if (abs(a[sel][col]) < EPS)
            continue;
        for (int i = col; i <= m; ++i)
            swap(a[sel][i], a[row][i]);
        where[col] = row;

        for (int i = 0; i < n; ++i)
            if (i != row) {
                double c = a[i][col] /
                    a[row][col];
                for (int j = col; j <=
                    m; ++j)
                    a[i][j] -= a[row][j]
                        * c;
            }
        ++row;
    }

    ans.assign(m, 0);
    for (int i = 0; i < m; ++i)
        if (where[i] != -1) ans[i] = a[
            where[i]][m] / a[where[i]][i];
    for (int i = 0; i < n; ++i) {
        double sum = 0;
        for (int j = 0; j < m; ++j)
            sum += ans[j] * a[i][j];
        if (abs(sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i = 0; i < m; ++i)
        if (where[i] == -1) return INF;
```

```
return 1;
}
```

Sendo n o número de linhas de A e m o de colunas, a complexidade desse código é $O(\min(n, m) \cdot mn)$

Dá pra resolver um desse mod 2 32 vezes mais rápido:

```
262530
int gauss(vector<bitset<N>> a, int n,
int m, bitset<N>& ans) {
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m
        && row < n; ++col) {
        for (int i = row; i < n; ++i)
            if (a[i][col]) {
                swap(a[i], a[row]);
                break;
            }
        if (!a[row][col]) continue;
        where[col] = row;

        for (int i = 0; i < n; ++i)
            if (i != row && a[i][col])
                a[i] ^= a[row];
        ++row;
    }
    // The rest of implementation is
    the same as above
}
```

8.2 Determinante e posto

Usaremos as ideias da subseção anterior. Calcularemos o determinante em $O(N^3)$

```
a8fc27
const double EPS = 1E-9;
int n;
vector<vector<double>>> a(n, vector<
double>(n));

double det = 1;
for (int i = 0; i < n; ++i) {
    int k = i;
    for (int j = i + 1; j < n; ++j)
        if (abs(a[j][i]) > abs(a[k][i])
            ) k = j;
    if (abs(a[k][i]) < EPS) {
        det = 0;
        break;
    }
    swap(a[i], a[k]);
    if (i != k) det = -det;
    det *= a[i][i];
    for (int j = i + 1; j < n; ++j)
        a[i][j] /= a[i][i];
    for (int j = 0; j < n; ++j)
        if (j != i && abs(a[j][i]) >
            EPS)
            for (int k = i + 1; k < n;
                ++k)
```

```

        a[j][k] -= a[i][k] * a[
            j][i];
    }
    cout << det;

```

8.2.1 Decomposição LU

Uma matriz A $n \times n$ inversível sempre admite uma fatoração $A = LU$ (em que L é lower e U é upper triangular, SPG com diagonal principal de $L = 1$). Ela é única se, e só se, todos seus (determinantes dos) menores principais são não nulos. Código em java:

```

19e17f
static BigInteger det(BigDecimal a[][],
    int n) {
    try {

        for (int i = 0; i < n; i++) {
            boolean nonzero = false;
            for (int j = 0; j < n; j++)
                if (a[i][j].compareTo(
                    new BigDecimal(
                        BigInteger.ZERO)) >
                    0)
                    nonzero = true;
            if (!nonzero)
                return BigInteger.ZERO;
        }

        BigDecimal scaling[] = new
            BigDecimal[n];
        for (int i = 0; i < n; i++) {
            BigDecimal big = new
                BigDecimal(BigInteger.
                    ZERO);
            for (int j = 0; j < n; j++)
                if (a[i][j].abs().
                    compareTo(big) > 0)
                    big = a[i][j].abs()
                        ;
            scaling[i] = (new
                BigDecimal(BigInteger.
                    ONE)).divide(big, 100,
                BigDecimal.
                    ROUND_HALF_EVEN);
        }

        int sign = 1;

        for (int j = 0; j < n; j++) {
            for (int i = 0; i < j; i++)
            {
                BigDecimal sum = a[i][j]
                    ];
                for (int k = 0; k < i;
                    k++)
                    sum = sum.subtract(
                        a[i][k].multiply
                            (a[k][j]));
            }

```

```

        a[i][j] = sum;
    }

    BigDecimal big = new
        BigDecimal(BigInteger.
            ZERO);
    int imax = -1;
    for (int i = j; i < n; i++)
    {
        BigDecimal sum = a[i][j]
            ];
        for (int k = 0; k < j;
            k++)
            sum = sum.subtract(
                a[i][k].multiply
                    (a[k][j]));
        a[i][j] = sum;
        BigDecimal cur = sum.
            abs();
        cur = cur.multiply(
            scaling[i]);
        if (cur.compareTo(big)
            >= 0) {
            big = cur;
            imax = i;
        }
    }

    if (j != imax) {
        for (int k = 0; k < n;
            k++) {
            BigDecimal t = a[j]
                ][k];
            a[j][k] = a[imax][k]
                ];
            a[imax][k] = t;
        }

        BigDecimal t = scaling[
            imax];
        scaling[imax] = scaling
            [j];
        scaling[j] = t;

        sign = -sign;
    }

    if (j != n - 1)
        for (int i = j + 1; i <
            n; i++)
            a[i][j] = a[i][j].
                divide(a[j][j],
                    100, BigDecimal.
                        ROUND_HALF_EVEN)
                    ;
    }

    BigDecimal result = new
        BigDecimal(1);
    if (sign == -1)

```

```

        result = result.negate();
    for (int i = 0; i < n; i++)
        result = result.multiply(a[
            i][i]);

    return result.divide(BigDecimal
        .valueOf(1), 0, BigDecimal.
        ROUND_HALF_EVEN).
        toBigInteger();
} catch (Exception e) {
    return BigInteger.ZERO;
}
}

```

É possível determinar o posto de uma matriz em $O(n^3)$ com eliminação gaussiana

```

19a3f4
const double EPS = 1E-9;

int compute_rank(vector<vector<double>>
    A) {
    int n = A.size();
    int m = A[0].size();

    int rank = 0;
    vector<bool> row_selected(n, false);

    for (int i = 0; i < m; ++i) {
        int j;
        for (j = 0; j < n; ++j) {
            if (!row_selected[j] && abs
                (A[j][i]) > EPS) break;
        }

        if (j != n) {
            ++rank;
            row_selected[j] = true;
            for (int p = i + 1; p < m;
                ++p)
                A[j][p] /= A[j][i];
            for (int k = 0; k < n; ++k)
            {
                if (k != j && abs(A[k][
                    i]) > EPS) {
                    for (int p = i + 1;
                        p < m; ++p)
                        A[k][p] -= A[j
                            ][p] * A[k][
                                i];
                }
            }
        }
    }
    return rank;
}

```

9 Miscelânea

10 Apêndice

10.1 Matemática

10.1.1 Constantes

```

/* Definitions of useful mathematical
   constants
* ME      — e
* MLOG2E  — log2(e)
* MLOG10E — log10(e)
* MLN2    — ln(2)
* MLN10   — ln(10)
* M_PI    — pi
* M_PI_2  — pi/2
* M_PI_4  — pi/4
* M_1_PI  — 1/pi
* M_2_PI  — 2/pi
* M_2_SQRTPI — 2/sqrt(pi)
* MSQRT2  — sqrt(2)
* MSQRT1_2 — 1/sqrt(2)
*/

#define ME
2.71828182845904523536
#define MLOG2E
1.44269504088896340736
#define MLOG10E
0.434294481903251827651
#define MLN2
0.693147180559945309417
#define MLN10
2.30258509299404568402

```

```

#define M_PI
3.14159265358979323846
#define M_PI_2
1.57079632679489661923
#define M_PI_4
0.785398163397448309616
#define M_1_PI
0.318309886183790671538
#define M_2_PI
0.636619772367581343076
#define M_2_SQRTPI
1.12837916709551257390
#define MSQRT2
1.41421356237309504880
#define MSQRT1_2
0.707106781186547524401

```

10.1.2 Fórmulas e Teoremas

Teorema de Lagrange: Todo inteiro positivo pode ser representado como soma de 4 quadrados.

Teorema 'Eureka' de Gauss: Todo inteiro positivo pode ser expresso como a soma de três números triangulares.

Teorema de Zeckendorf: Todo inteiro positivo tem uma representação única como soma de números de Fibonacci tal que não existem dois números iguais ou consecutivos.

Fórmula de Euclides para ternas pitagóricas: Ternas pitagóricas primitivas são todas da forma

$$(n^2 - m^2, 2nm, n^2 + m^2)$$

em que $0 < m < n$, $(n, m) = 1$ e $2|nm$.

10.1.3 Números primos